Learning to Formulate: Agentic Hardware–Software Co-Design of AI Infrastructure through Progressive Optimization and Verification

Li Shang Ding Yi [further authors]

National University of Singapore

Revised draft, October 18, 2026.

Abstract

Designing the hardware and the software of AI infrastructure together is held back less by the size of the design space than by the absence of a formulation in which that space can be searched and trusted. Written all at once, whole-stack co-design is a mixed-integer program no solver can crack; written layer by layer, it gives up the cross-layer interactions where the largest gains lie. This paper proposes that a language model should *write the formulation*, one refinement step at a time, as a *template*: a contract in which the constraints that bound the solver's search are the same conditions under which a verifier proves every admitted design correct. Templates make three things possible. The model writes them, and the solver and the verifier are bound by them without further help from the model. The verifier certifies the whole family a template admits, once, before the solver selects a design, so that a race the template could produce is not a bug to be found afterward but a design the solver cannot choose. And the flow descends the stack from the workload to the chip microarchitecture through a sequence of such templates, co-designing software and hardware at every step. We implement the flow and evaluate it on two hardware models, the open-source Ventus RISC-V GPGPU and a model of the NVIDIA GB10 calibrated on a DGX Spark, running large-language-model workloads. Across refinement steps the verifier catches 31 injected faults and 3 deadlocks in 78 structural variants with a replayed witness for each; progressive checking issues 27 solver queries at depth five instead of 92; and designing the chip and the software together beats designing either alone, by up to [best joint-vs-single-side speedup across the two models] on the worst-case scenario.

1 Introduction

Progress in AI is increasingly limited not by the models but by the infrastructure that runs them. Every gain in capability has been paid for with more computation, memory and data movement, and the cost, energy and availability of AI infrastructure now decide what can be trained and served. Designing that infrastructure is a co-design problem that spans the entire stack: the workload at the top, the software that executes it, from serving systems through compilers and kernel libraries, and the chip underneath, down to its microarchitecture. The layers are deeply coupled. Whether a kernel runs fast depends on the size of the register file and the on-chip memory; whether that memory is worth its silicon depends on whether the software can reuse data through it. The largest efficiency gains come from exactly these interactions. The layers also move on mismatched timescales: a new model or inference technique can change the demands on the hardware within months, while a new chip takes years to design, verify and bring to production.

Industry copes by taking the stack apart. Chips and software are developed on separate tracks, behind interfaces frozen early. A chip is architected against representative workloads, which by the time it ships are

the workloads of the past, and the software is then written to fit whatever chip it is handed. This separation is what has made modern AI systems buildable, but it fixes in advance which decisions may interact, and it commits to each decision before the decisions that depend on it are known. A tile size is chosen for a register file that was sized for last year's tiles. The cross-layer combinations where specialization pays off are never explored, not because they are hard to build, but because no step of the process ever considers them.

The natural response is to remove the boundaries: write the whole co-design down as a single optimization problem and turn a solver loose on it. This does not work, and the reason is fundamental. Written all at once, with every choice of tiling, data movement, scheduling, instruction selection and hardware resource exposed together, co-design becomes an enormous mixed-integer program whose structure is far too weak for any solver to crack. The obstacle is neither the hardware nor the algorithm but the *formulation itself*. Layer-wise engineering is a fixed, hand-made formulation that keeps the problem solvable by giving up its most valuable interactions; the monolithic model keeps every interaction and gives up solvability. Neither discovers the formulation in between, one that exposes the interactions that matter while remaining small enough to solve. The real question is not how to solve a fixed formulation but how to find one worth solving.

Three claims. This paper proposes that the formulation should be *written*, step by step, by a language-model agent, and that what it writes should be a contract that an optimization solver and a formal verifier can both read. Three claims carry the paper, and the paper is organized around them.

1. **A language model writes templates.** A *template* is a contract with six parts: a scope, decisions, constraints, a cost model, correctness preconditions and the conditions that bind them (§2.3). The agent writes it from the coarse formulation, a catalog of the structure that physics gives the co-design space (§3), and the templates it has certified before. The solver and the verifier then read the template without further help from the model (§2.4).
2. **A verifier certifies the whole family a template admits**, once, before the solver selects a design (§5). The conditions that bound the search are the preconditions of the proof, so the designs the solver may choose are exactly the designs the verifier will accept. Per candidate, what remains is a constant-time check.
3. **The flow co-designs chip and software down the stack** through a sequence of templates, each refining one coarse data transaction into finer ones (§4). The optimizer never writes down the monolithic program, and the verifier owes one local proof per step; local proofs compose back to the workload. On two hardware models, the ladder *baseline* $\rightarrow$ *software only* $\rightarrow$ *hardware only* $\rightarrow$ *sequential* $\rightarrow$ *joint* shows where co-design pays and where it does not (§7).

Where this sits among verifiers. Checking that a kernel, a compiler transformation or an optimization reformulation is correct is well studied, and the recent kernel verifiers come closest to our setting: ProofWright proves an LLM-written CUDA kernel memory-safe and race-free [103], and Volta proves an optimized GPU kernel equal to a reference [104]. Each verifies *one artifact after it exists*. Our verifier is asked a different question at a different time: is *every* design this template can produce correct, asked *before* the solver runs, with the answer fed back as the constraint the solver must respect. A race the kernel verifiers would catch in the finished kernel is, in our flow, a design the solver cannot select, because the condition that excludes it is the same line on both faces of the contract. Section 8 makes this comparison line by line.

Evaluation. We implement the flow and evaluate it in three parts. First, one template end to end (§7.2): its search constraints and correctness conditions coincide, the solver's optimum is certified in milliseconds, and a contract that is too strong or too weak costs exactly what the theory predicts. Second, the flow on two hardware models (§7.3): the Ventus open-source RISC-V GPGPU and a model of the NVIDIA GB10 calibrated on a DGX Spark, on which the same studies are run and the same ladder of strategies is reported,

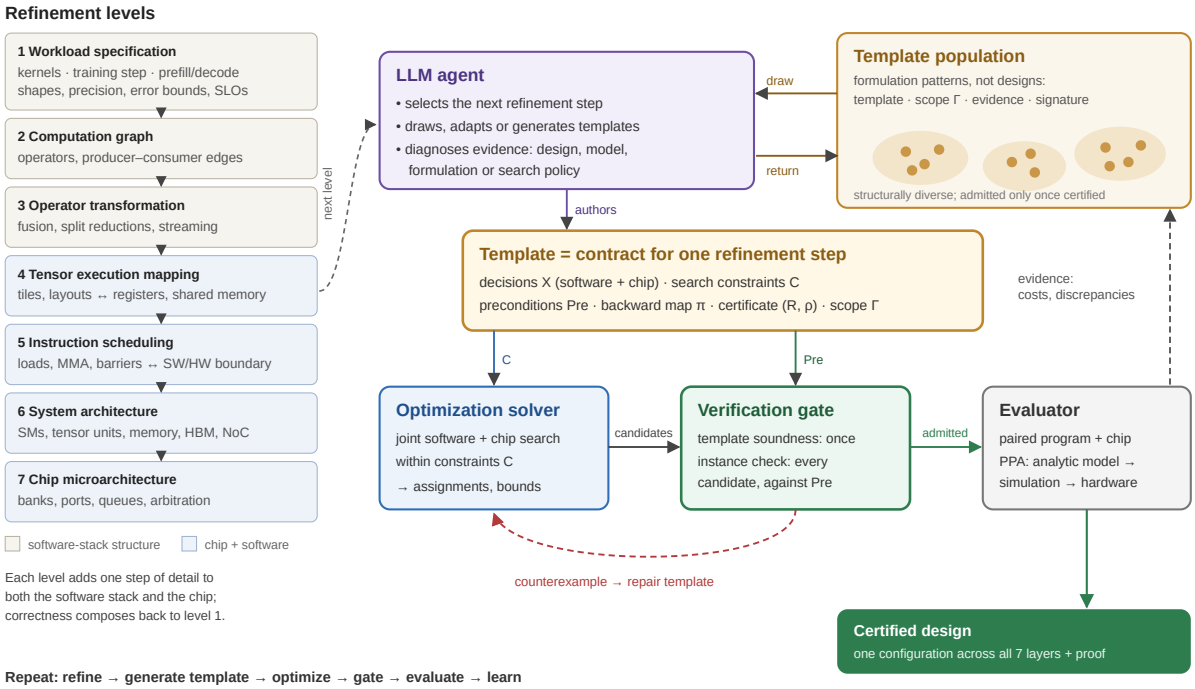


Figure 1: The agentic progressive refinement flow. The design moves down the refinement levels from the workload specification to the chip microarchitecture; each level adds detail to both the software stack and the chip. At each step the agent draws a template from the population or writes a new one. The template is a contract: its search constraints C bound the solver's joint search over software and chip decisions, and its preconditions Pre are what the verification gate checks for every candidate. Only admitted designs reach the evaluator. Counterexamples return to the agent as template repairs, and costs and discrepancies return to the population as evidence. The output is one configuration across all layers, together with a proof that it computes the workload.

so that a co-design gain can be seen to be a property of the workload or of one chip. Third, the cost of the gate (§7.4). Results appear where they illustrate a claim; the limits of the evidence are collected once, in §9.

Organization. Section 2 presents the flow, defines the template as a contract and shows the language model writing one. Section 3 explains where templates come from. Section 4 develops progressive refinement, Section 5 the verification side and Section 6 the optimization side. Section 7 reports the evaluation, Section 8 positions the work, and Section 9 states what the evidence does and does not support. Proofs and the structure catalog are in the appendices.

2 The Flow and the Template

This section presents the flow end to end. We state what is being designed (§2.1), walk through one iteration of the loop (§2.2), define the template as a contract (§2.3), show the agent writing one (§2.4) and characterize the weakest contract (§2.5). Figure 1 shows the flow as a whole.

2.1 What is designed, and what it must compute

Workloads. The flow takes as input a large-model workload that the chip and its software must serve: a single kernel at given tensor shapes, a training step, or an inference workload such as prefilling a batch of prompts followed by a bounded number of decoding steps. A design usually serves several workloads at once; an inference chip must handle both prefill and decode, which share weights but stress the chip very differently. We call each workload a *scenario* and let $\mathscr{W}$ be the finite set of scenarios. Each scenario has a *workload contract* with parameters $p \in \mathscr{P}$ (tensor dimensions, batch size), admissible runtime inputs $U(p)$, and a set $W(p,u)$ of acceptable behaviors for each input $u \in U(p)$. A behavior records visible events, final outputs, state updates and completion status; units of work terminate, and the contract excludes deadlock. The workload contract is the gold reference of the flow. It is specified independently of any optimization model, so that omitting a constraint from a formulation can never silently erase a requirement.

Hardware resource library. The chip is assembled from a library $\mathscr{L}$ of configurable components: streaming multiprocessors, tensor units, register files, shared-memory blocks, cache slices, memory controllers, interconnect. Each component type c has a configuration θ_c from a finite set Θ_c (capacity and banks for a shared-memory block, registers and ports for a register file, the matrix shape per cycle for a tensor unit) and a parameterized model of its performance $\mathrm{Perf}_c(\theta_c)$, area $A_c(\theta_c)$, access energy $E_c(\theta_c)$ and static power $P_c^{\mathrm{stat}}(\theta_c)$, analytical or fitted to synthesis and simulation data. These models are what allow the optimizer to evaluate a chip configuration without building it.

Designs. The design space is organized by the seven layers of the stack, $\lambda = 1,\dots,7$: workload specification, computation graph, operator transformation, tensor execution mapping, instruction scheduling, system architecture and chip microarchitecture. A *design* $d = (x_1,\dots,x_7) \in \mathscr{X}_1 \times \cdots \times \mathscr{X}_7$ selects one value for every decision at every layer, and so fixes one concrete configuration of the entire software–hardware stack. The decisions at the last two layers determine the chip configuration $h = \{(n_c,\theta_c)\}_{c\in\mathscr{L}}$, an instance count and a configuration for each component type. The software–hardware boundary is itself a decision, b, which selects for mechanisms such as data transfer whether each is realized in software, through a programmable instruction or by dedicated hardware. When a design serves several scenarios, the chip and its boundary are shared and the software-side decisions may differ per scenario. Section 4 indexes the refinement levels below the workload specification by $\ell = 0,\dots,5$; level ℓ is layer $\ell+2$.

Objectives. A design is judged by performance, power and area computed from the component models and the software running on the chip. Area is $A(h,b) = A_0 + \sum_c n_c A_c(\theta_c) + A_B(b)$. The execution time $L_w(d)$ of scenario w is the cycles its software takes on the chip divided by the clock; the average power is static power plus energy per access times access counts over L_w. The flow minimizes a PPA objective $J(d)$ subject to an area budget, a power envelope per scenario and domain requirements $g_r(d) \le 0$. For a kernel or a training step a natural objective is the worst normalized latency across scenarios, $J(d) = \max_w L_w(d)/L_w^{\mathrm{ref}}$; for serving, tokens per second per watt or per unit area subject to a per-user token rate and a bound on time to first token. Every one of these quantities couples software and chip decisions.

2.2 One iteration of the flow

The flow descends the stack from the computation graph to the microarchitecture. Its state after iteration k consists of a partially refined design, the active formulation $\mathscr{F}_k$ describing the decisions currently open, the evidence gathered so far, and the template population $\mathscr{Q}_k$ (§3.1). Five participants act on that state: the **language-model agent**, which reasons about the problem in domain terms, decides what to refine next, and writes, adapts or retrieves templates; the **template**, the contract the agent writes for the step; the **optimization solver**, which searches the formulation assembled from the active templates jointly over software and chip

variables; the **verifier**, which establishes against the template's own conditions that every admitted design still computes what the level above demanded, returning proofs, replayed counterexamples or an explicit *unresolved* verdict; and the **evaluator**, which realizes selected designs as a paired software implementation and chip configuration and measures them through analytical models, simulation or hardware.

One iteration proceeds in seven steps.

1. **Select the refinement.** The agent chooses the next boundary to cross, for example from logical tensors to tiles held in on-chip memory, and the region of the design it affects.
2. **Retrieve or write the contract.** The agent queries the population for templates whose scope covers the current context, instantiates a match, adapts a near match, or writes a new template (§2.4).
3. **Certify the template if it is new.** A template passes a one-time soundness check over its whole declared family (§5.5). A failure returns a replayed counterexample, and the agent repairs the template or abandons it.
4. **Optimize.** The solver searches the formulation formed by the active templates and the shared chip variables (§6).
5. **Gate each candidate.** Every selected assignment is checked against the instance-level conditions of its templates. This check is cheap because the expensive argument was made once, in step 3.
6. **Evaluate.** Admitted candidates are realized and measured. Feasibility failures, ranking disagreements between model and measurement, and search difficulty are each attributed to their source (§6.3).
7. **Learn.** The agent updates cost coefficients and constraints, records the evidence with its scope, and returns the template to the population. Proofs whose premises are unchanged are retained (§5.6).

The loop repeats, refining further down the stack or revisiting an earlier boundary when new evidence warrants it, until the budget is spent. It returns the best admitted design, the bounds and termination status that support it, and the composed correctness certificate that connects it to the workload.

2.3 The template as a contract

A refinement template governs one boundary, from level ℓ to level $\ell+1$. It introduces one new design object and everything needed to optimize and verify it. Formally, a template is a tuple

$$\mathsf{T} = \big(O,\, X,\, C,\, \mathrm{Pre},\, P,\, \pi,\, \Sigma,\, \hat{J},\, \Gamma\big),$$

whose components are listed in Table 1.

Scope, searched and frozen. One rule decides what belongs in the scope and what is a decision: *a quantity is in the scope only if nothing at this step can change it*. The shared-memory capacity of a chip whose memory hierarchy is already fixed is in scope; the number of threads per block is not, even when the template fixes it to one value, because a later template could choose differently. Decisions therefore come in two kinds. *Searched* decisions are what the solver chooses. *Frozen* decisions are choices this template made and fixed, which a later template may reopen (*unfreeze*). Derived quantities, such as the number of reduction tiles, follow from the two. The distinction matters because it says which lines of the contract discharge which preconditions, and therefore what must be re-proved when a choice is reopened (§5.5).

The template is a contract among three parties: the agent that writes it, the solver that must respect it and the verifier that enforces it. Three conditions bind them.

(K1) The search face implies the verification face. Every assignment the solver may choose satisfies the preconditions: for all $p \in \Gamma$ and $x \in X$, the constraints $C(p,x)$ *together with the frozen choices* imply $\mathrm{Pre}(p,x)$.

Part	Meaning	Who reads it
O	The new design object: a shared-memory slot, an asynchronous transfer, a buffer, a resource binding	all
X	Its decisions and domains, software-side and chip-side, split into *searched* and *frozen* (below)	solver, agent
C	Search constraints over X: the feasible region the solver may choose from	solver
Pre	Preconditions under which the object preserves the computation of level ℓ	verifier
P	Operation constructors that give the object an executable meaning at level $\ell+1$	verifier, evaluator
π	The backward map that interprets a level-$(\ell+1)$ design as a level-ℓ design	verifier
Σ	A certificate schema: a state relation R and a progress rank ρ (§5.2)	verifier
$\hat{f}$	Cost-model terms for the object, with the assumptions they rest on	solver, evaluator
Γ	Scope: the workload family, the chip facts and the refinement step under which the template is valid	agent, verifier

Table 1: The parts of a template and the party that reads each.

This is what allows the solver's choice to be trusted without re-deriving its correctness. When C is exported to a solver through auxiliary variables or big-M encodings, the exported predicate must also imply C (§5.5).

(K2) The preconditions are sufficient. For every parameter in scope, every feasible coarse design and every assignment satisfying Pre, the refined design satisfies the local refinement obligations of §5.2 under the certificate schema Σ. This is proved once, for the whole family.

(K3) The contract is not empty. A witness assignment exists for every supported parameter, so that a sound template cannot appear correct merely because it admits nothing.

A contract satisfying K1–K3 is *certified*. Checking an individual solution then reduces to confirming that it lies in C and that its context lies in Γ. This split between a one-time family-level argument and a cheap per-instance check is the two-tier trust boundary at the center of the flow.

Example: the shared-memory tiling template. Listing 1 shows a complete template, the one evaluated in §7.2. It refines a GEMM $C \mathrel{+}= AB$ from logical tensors to tiles staged through on-chip shared memory. Its new object is a shared-memory *slot*: a thread block copies a $T_m \times T_k$ tile of A and a $T_k \times T_n$ tile of B into one of S slots, waits at a barrier, computes from the slot, and may wait at a second, trailing barrier ($\beta = 1$) before the slot is refilled. Loads for reduction tile $p+P$ are issued while tile p is being consumed, so the prefetch distance P overlaps data movement with computation.

Three kinds of line. Read from top to bottom, the listing is the whole agreement, and three kinds of line appear. *Scope* lines are facts no step can change. *Frozen* lines are choices this template made and a later template may reopen; the chip-level template of §7.3 reopens M_{blk}, and a register-tiling template reopens $a_{\max}$. *Searched* lines are what the solver decides. The verifier's preconditions are discharged by exactly one of the last two kinds, and the listing says which. The solver sees the decisions, the constraints C1–C3 and the cost; it never needs to know what a race is. The verifier sees the preconditions, the program, the backward map and the certificate; it never needs to know what is being optimized. The agent sees the scope and the frozen choices: the template may be reused for any GEMM shape under the same chip facts, and must be re-examined when a frozen choice is reopened or a chip fact becomes a decision.

One line, four readers. The constraint $P \le S - 2 + \beta$ is where the contract becomes visible. With a trailing barrier, the slot refilled in phase p last held tile $p+P-S$, which every warp finished before that barrier; this requires $P \le S-1$. Without it, only the leading barrier of phase $p-1$ orders a slow warp's reads, so the overwritten tile must have been consumed by phase $p-2$, which requires $P \le S-2$. Each party reads the

```
template SharedMemoryTiling
  refines    GEMM at the operator level -> shared-memory-tiled GEMM
             at the tensor-mapping level
  object O   shared-memory slot: one Tm x Tk tile of A and one Tk x Tn tile of B

scope Gamma                                                  [agent, verifier]
  workload   C[M,N] += A[M,K] B[K,N],  any M, N, K >= 1,  FP32,  e = 4 B/elem
  chip       N_SM = 128;  M_blk = 96 KiB shared memory per block;
             R_SM = 65,536 registers per SM;  r_max = 255 registers per thread;
             F_SM = 625 GFLOP/s per SM;  B_DRAM = 1.8 TB/s;  t_bar = 40 cycles
  semantics  barrier: no thread passes until all arrive, and shared-memory
             writes before it are visible after it.
             load: a tile load completes before the next barrier.

decisions X
  searched                                                          [solver]
    Tm, Tn in {16, 32, 64, 128, 256}    Tk in {8, 16, 32, 64}
    S in {1, 2, 3}    P in {0, 1, 2}    beta in {0, 1}
  frozen  (fixed by this template; a later template may unfreeze)  [template]
    n_t = 256 threads per block;  a_max = 128 accumulators per thread;
    grid = ceil(M/Tm) x ceil(N/Tn);  output tiles disjoint;
    guard (index < extent) on every tail;  leading barrier in every phase
  derived
    KT = ceil(K/Tk);  smem = S e (Tm Tk + Tk Tn);  acc = Tm Tn / n_t

constraints C                                                       [solver]
  C1  S e (Tm Tk + Tk Tn) <= M_blk           # shared-memory capacity
  C2  1 <= Tm Tn / n_t <= a_max              # each thread owns >= 1 output,
                                             #   <= its accumulator budget
  C3  P <= S - 2 + beta                      # slot lifetime   (== Pre.lifetime)
  check  n_t r_max <= R_SM                   # register file holds one block

preconditions Pre                                  [verifier]  discharged by
  lifetime    P <= S - 2 + beta                                C3 (searched)
  readiness   a slot is read only after its load's barrier     barrier (frozen)
  coverage    every output element lies in exactly one tile    grid (frozen)
  exact-once  tiles are disjoint; no product is added twice    disjoint (frozen)
  no-phantom  no access outside [M, N, K]                      guard (frozen)

program P (level l+1)                                  [verifier, evaluator]
  for each block (i, j):
    for u in 0 .. P-1:    load k-tile u -> slot u mod S
    for p in 0 .. KT-1:
      if p+P < KT:        load k-tile p+P -> slot (p+P) mod S
      barrier                                          # leading
      compute C[i,j] += slot (p mod S)
      if beta: barrier                                 # trailing

backward map pi                                                     [verifier]
  forget the grid, the slots and the schedule  =>  C = A B over (M, N, K)

certificate Sigma                                                   [verifier]
  relation R   slot (u mod S) holds A[:,u], B[u,:] from the completion of its
               load until the last read of k-tile u
  rank rho     number of k-tiles not yet consumed
  obligations  coverage, exact-once, no-phantom  (index; symbolic extents)
               readiness, lifetime               (schedule; all interleavings)
               agreement: replay yields exactly the terms A[i,k] B[k,j]

cost J^                                                    [solver, evaluator]
  T >= T_comp,  T >= T_mem,  T >= T_comp + T_mem if P = 0
  assumptions (steer, never proved): one block per SM; no L2 reuse;
               tails cost as full tiles
```

Reader	How it reads $P \le S-2+\beta$	What follows
Solver	A linear row over the menu indicators y_p^P, y_s^S and the barrier bit: $\sum_p p\, y_p^P - \sum_s s\, y_s^S - \beta \le -2$	Unsafe schedules are outside the feasible region, so the optimum can never be one of them
Verifier	The precondition of the lifetime obligation: in every admissible interleaving of loads, barriers and computes, no slot element is overwritten before its last read	Proved once for the family (K2); each instance is then certified by evaluating the line itself
Agent, writing	The lifetime structure from the catalog (§3) for a ring of S slots with prefetch distance P; the agent must also supply the obligation it discharges	A template that states the constraint without the obligation is not a contract and is not admitted
Agent, reusing	Evidence stored with the template: the condition is exact on all 24 checked combinations of S, P and β (§7.2)	Precondition loss is zero; a stronger line costs 1.34%, and a weaker one admits a race (Figures 3 and 4)

Table 2: One line of the contract, read by each party.

same line differently (Table 2).

Because C3 is literally the precondition, K1 holds for lifetime by construction; the other four preconditions hold under every assignment because the choices that discharge them are frozen. K2 is the lifetime lemma together with the index obligations, proved once over the family. K3 holds because the smallest tiles with $S = 1$, $P = 0$ and $\beta = 1$ satisfy C1–C3 for every shape in scope. Figure 2 shows the same contract schematically.

2.4 Writing the template

The listing above is what the language model writes. In the flow, the agent receives the coarse formulation, the structure catalog of §3 and the population's nearest template, and produces a contract in this form. The solver and the verifier then read it without further help from the model. The claim is not that the agent writes correct templates; it is that the verifier decides, and that what the agent writes is in a form the verifier can decide. Two kinds of evidence show that this is feasible.

Repairing a template. The shared-memory tiling template was first written by hand with chip limits and frozen software choices mixed in one scope block, the accumulator bound written as the constant 32,768, and a single precondition, lifetime, as if C3 alone implied correctness. Given that version and the scope rule of §2.3, the agent (Claude, October 2026) separated the lines by the rule, rewrote C2 as $1 \le T_m T_n / n_t \le a_{\max}$ so that the two frozen choices hidden in the constant became visible, added the register-file check $n_t r_{\max} \le R_{\mathrm{SM}}$, listed all five preconditions with the line that discharges each, and recomputed the cost model from the template's own definitions. The verifier re-certified the result: the same 722 feasible designs out of 1,800, the same optimum on the three GEMM shapes of §7.2, and 12 of 12 obligations proved for each selected design. The repair changed what the contract *says*, not what it *admits*, which is the kind of change the population needs before a template can be reused by another agent. It is one template and one agent run.

Writing a template. The second refinement of the GEMM, from shared-memory tiles to register tiles in which each thread holds an $r_m \times r_n$ block of outputs in registers and reads operands from the slot, was written by the agent given the repaired tiling template as its one example and the catalog. The template it wrote unfreezes $a_{\max}$, adds the searched decisions r_m, r_n and the derived accumulator count $r_m r_n$, and proposes two new preconditions: *ownership* (each output element is owned by exactly one thread, discharged by the thread-to-tile map it freezes) and *register readiness* (a register operand is read only after its load from the

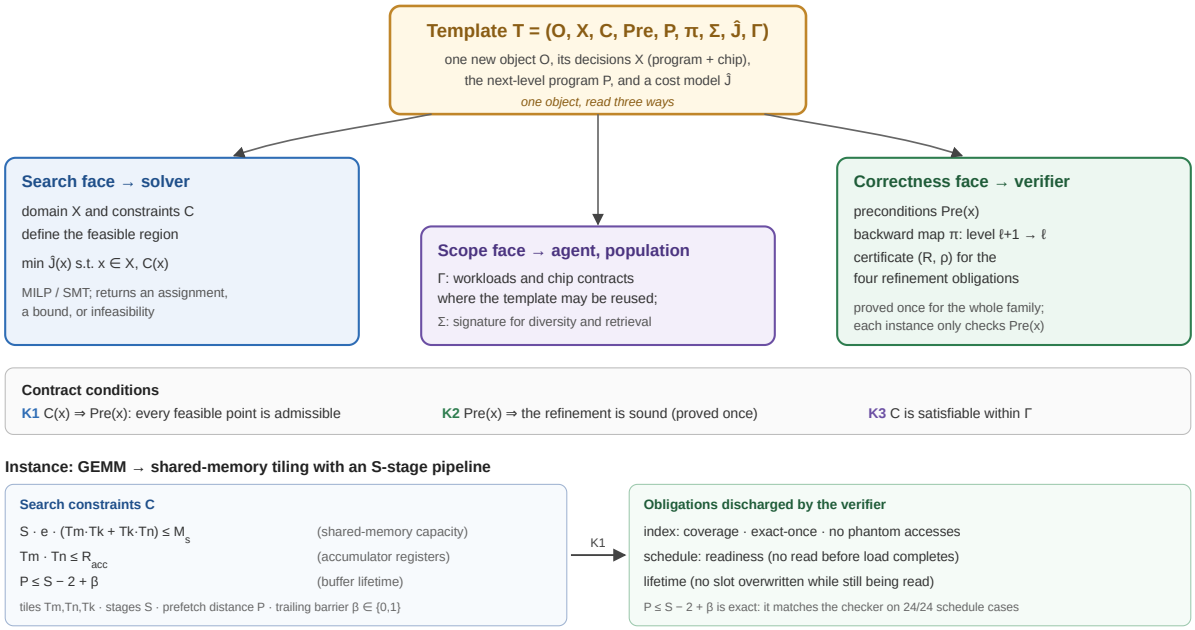


Figure 2: A template is one object read three ways. The solver reads its search face (searched decisions, constraints C, cost $\hat{J}$); the verifier reads its correctness face (preconditions Pre, backward map π, certificate (R, ρ)); the agent reads its scope Γ and frozen choices. Conditions K1–K3 bind the three readings. Bottom: the shared-memory tiling instance, in which one searched constraint and four frozen choices discharge the five obligations the verifier proves. [Figure to be redrawn with the four blocks of Listing 1: scope, searched, frozen, preconditions.]

slot). [Report, from the agent run: the template as written; which of its proposed preconditions the family check certified as sound and which were refuted with a replayed counterexample; whether the sound ones were exact on the finite family; the number of repair rounds; and the solver's optimum under the certified template.] A refuted first attempt that is repaired from the counterexample exercises the loop of §2.2 and is the more informative outcome. Section 7.2 carries this template through the same three tables as the first.

2.5 The weakest contract

For a template T and parameters p, let $\mathrm{Safe}_{\mathsf{T}}(p)$ be the set of assignments whose refined designs actually preserve the coarse behavior. This is the weakest precondition, in Dijkstra's sense [12], of behavior preservation with respect to the template's operations. A precondition is *sound* if $\mathrm{Pre} \subseteq \mathrm{Safe}$ and *exact* if equality holds. Let $J_k^\star(\mathrm{Pre})$ denote the best objective attainable in the active formulation when the template's assignments are restricted to Pre.

Definition (precondition loss). The precondition loss of a sound precondition is $\Delta_{\mathrm{pre}} = J_k^\star(\mathrm{Pre}) - J_k^\star(\mathrm{Safe}) \geq 0$.

Proposition 1 (monotonicity of the contract). If $\mathrm{Pre} \subseteq \mathrm{Pre}' \subseteq \mathrm{Safe}$, then $J_k^\star(\mathrm{Pre}') \leq J_k^\star(\mathrm{Pre})$, and an exact precondition has zero precondition loss. If instead $\mathrm{Pre}' \not\subseteq \mathrm{Safe}$, the formulation admits a design that violates the workload contract, and its optimum need not be attainable by any correct design.

Proof. The first claim follows because the feasible set only grows. The second follows because some $x \in \mathrm{Pre}' \setminus \mathrm{Safe}$ is feasible, and nothing prevents the objective from favoring it. □

The proposition is elementary, but it states the central tension precisely. Verification wants the honest

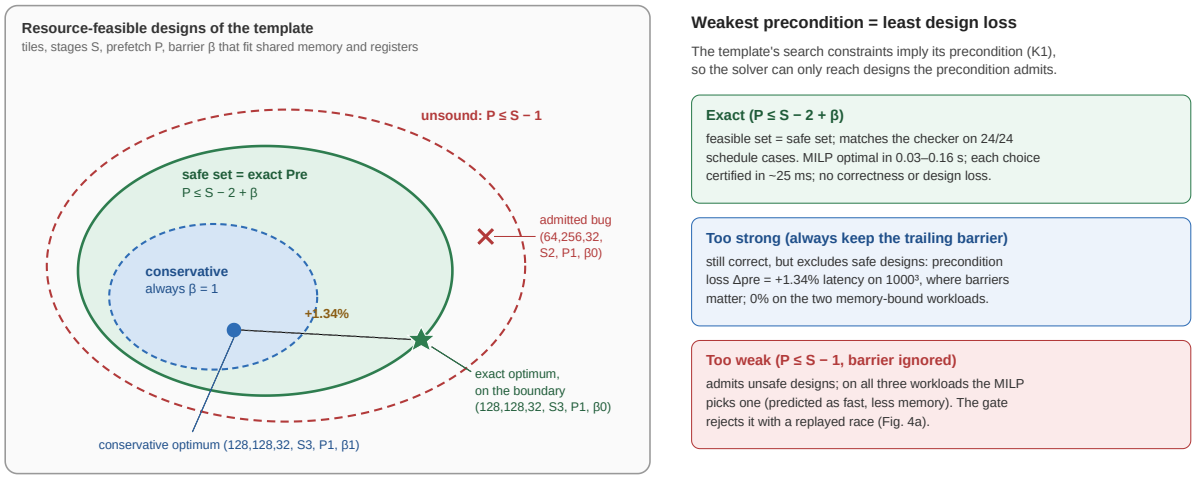


Figure 3: Weakest precondition as least design loss, for the shared-memory tiling template. The exact precondition equals the safe set and contains the optimum on its boundary. A conservative precondition (always keep the trailing barrier) is sound but excludes that optimum, a precondition loss of 1.34% on the 1000^3 GEMM. An unsound precondition ($P \leq S-1$) admits an unsafe design that the solver then selects, and the gate catches.

boundary of correctness, and optimization wants the largest correct search space. Both want the same set. Exact preconditions are rarely available in closed form, so the flow treats them as a target: the agent proposes Pre, and the verifier probes both directions, confirming soundness on the declared family and, where the family is finite, testing whether the rejected assignments are all genuinely unsafe. Figures 3 and 4 show both failure modes for the shared-memory tile. The exact precondition $P \leq S-2+\beta$ contains the solver's optimum on its boundary. Always keeping the trailing barrier is sound but too strong, and costs 1.34% of latency on a ragged GEMM. Dropping the barrier from the precondition is too weak: the solver then selects a design with a race, which the gate rejects.

3 Where Templates Come From

Section 2 says what a template is and how it is checked. It does not say how the agent knows what to write. If the co-design space were an arbitrary combinatorial problem, nothing could guide it. It is not. Its structure comes from physics, and that structure is what the agent recognizes and writes into each template.

The physical origin. Three facts govern every computer system. Memory can be large, but large means far away, and far means slow and costly to reach. Computation can be fast, but it is useless without data. Moving data is physics, and no design bends it. The memory hierarchy exists because of these facts, and it is a fixed ladder of cost rather than a design choice to search over: DRAM, then L2, then shared memory, then registers, each roughly an order of magnitude better than the level below it in bandwidth, latency and energy per byte. Two consequences follow. Data movement dominates cost, and it dominates in a known order.

Three classes of structure. From these facts follow a small number of recurring structures, cataloged in Appendix A. They are easier to use when organized by *what each does to the search*, because that is what determines the formulation move the agent writes:

- **Structures that collapse the objective** tell the agent *what to optimize*: bottleneck dominance (one

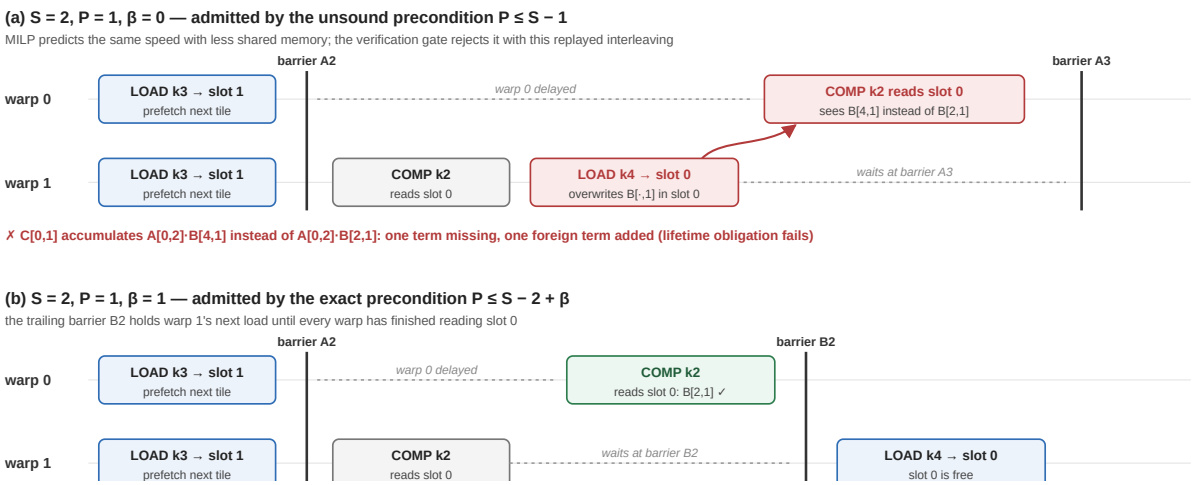


Figure 4: The counterexample the gate replays for the unsound precondition, with $S = 2$ slots and prefetch distance $P = 1$. (a) Without a trailing barrier ($\beta = 0$), warp 1 finishes reading slot 0, crosses no barrier, and prefetches k-tile 4 into slot 0 while warp 0 is still waiting to read k-tile 2 from it. Output $C[0,1]$ then accumulates $A[0,2]\,B[4,1]$ instead of $A[0,2]\,B[2,1]$. (b) With the trailing barrier ($\beta = 1$), which the exact precondition requires for this design, the load waits until every warp has finished reading the slot.

resource is far costlier than the rest, so its utilization is a near-sufficient statistic), concavity (performance saturates at a roofline knee [102], so optima lie on constraint boundaries) and monotonicity (at fixed area, more of a resource never hurts, which gives valid one-sided bounds).

- **Structures that collapse the variables** tell the agent *what is independent*: near-decomposability [101] (interactions across the stack are local, and the hierarchy itself cuts chains, so that shared memory separates DRAM from the registers), scenario decomposition (prefill and decode share a chip but want different software, so the chip is the complicating variable) and low effective dimension (only the knobs that touch the bottleneck matter).
- **Structures that collapse the enumeration** tell the agent *which points to skip*: symmetry, discrete menus and dominance.

The agent's task when it writes a template is to answer these three questions for the current refinement step. A template that answers none of them has no reason to exist.

The structure is the template. Each structure maps to two things at once, and this is what connects the catalog to the contract. On the search face it is a *move in the formulation*: a surrogate objective, a decomposition, a bound, a frozen decision, a symmetry-breaking constraint. On the correctness face it is an *obligation*: the structure holds only in a *regime*, for instance "this region is DRAM-bound", and the agent owes the verifier evidence that the regime holds wherever the move is relied upon. The obligations differ in cost. Symmetry, monotonicity and discrete menus are algebraic and are proved once per template. Bottleneck dominance and near-decomposability may cost a sub-solve. Section 5.7 draws the consequence: these are *performance* obligations, a different kind from the correctness obligations of §5.2, and they never gate a design.

One structure end to end. Consider the bottleneck. *Detection* is cheap: the agent runs the coarse cost model once and reads each resource's utilization against its roofline ceiling. Suppose DRAM sits at 95% and the compute units at 40%. The output is a claim: this region is DRAM-bound. *The move* rewrites the formulation in two ways. It replaces the full performance objective with a surrogate, DRAM bandwidth utilization, which is tighter and simpler; and it freezes every decision that does not touch DRAM traffic, such as register port

count and tensor-unit shape, because by low effective dimension they are nearly free here. A problem over twenty coupled decisions becomes a problem over the three or four that move bytes: tiling, shared-memory residency and the transfer schedule. The frozen lines of Listing 1 are this move written down. *The obligation* is that DRAM remains dominant across the feasible region of this subproblem. A tiling choice could relieve DRAM and hand the bottleneck to shared memory, flipping the ranking of branches. The verifier therefore establishes a bound: even the most DRAM-relieving design in the branch leaves DRAM above the next level. If it cannot, the branch straddles a regime boundary and the agent splits it.

Regime flips are refinement steps. When the evaluator reports that the bottleneck has moved, that is not a failure of the template; it is the signal to refine. The bottleneck has moved one rung down the hierarchy. The agent re-detects, finds shared memory pinned, and writes the next template one level lower, with a new surrogate and a new precondition. The register-tiling template of §2.4 is exactly this step: the tiling template's cost model declares the 1000^3 GEMM compute-bound at the optimum, and the structure that collapses the next objective is register reuse. The regime boundaries of the search space are therefore the refinement steps of §4, seen through cost: the agent always formulates around the data movement that currently dominates, and physics tells it what to refine next.

3.1 The template population

Templates are not written once and filed away. The agent maintains them as a population $\mathscr{Q}_k = \{(\mathsf{T}_i, \Gamma_i, \mathscr{E}_i, \varsigma_i)\}$, in which each member pairs a template with its scope, its accumulated evidence $\mathscr{E}_i$ (soundness proofs, certified instances, measured outcomes, counterexamples and their causes, and for each structural assumption its strength and where it was observed to flip) and a structural signature ς_i summarizing the kind of design it produces: its algorithmic organization, reuse direction, storage placement, boundary mode and the chip resources it relies on. Members of the population are formulation patterns, not solved designs. In each iteration the agent draws candidates whose scope covers the current context, adapts or recombines them, and returns those that survive verification, refined by what the iteration revealed.

One rule governs admission: **a template enters the population only after K1–K3 have been established within its declared scope.** Uncertified proposals may be explored, but their results are labeled as such and are never composed into a correctness claim. Two further rules govern what is kept. Knowledge carries its conditions: a statement such as "fusing these two operators is faster" is stored with the chip configuration and workload under which it was observed, and a later change to the chip reopens it (§7.3 shows why). Diversity is structural: retention favors members with distinct signatures over parameter variations of the same design, so that an alternative that looks unpromising on today's chip remains available when the chip changes. Over many problems the population accumulates not only formulations but *structure–regime pairs*: which structures hold for which workload families on which chips. This is the sense in which the agent improves at formulating, and because every template passes the gate before it is admitted, the improvement rests on proof and measurement rather than on the agent's own judgment. Transfer across workloads is not evaluated in this paper beyond the one measurement reported in §7.3.

4 Iterative Optimization through Progressive Refinement

A workload, at the top of the stack, describes its data movement only coarsely: an output *is* a sum of products, and an activation *flows* from one operator to the next. A running chip moves data in fine-grained transactions: a tile is copied into a numbered slot, a transfer is issued and later completes, a request waits in a queue for a bank port. Progressive refinement is the passage from the first description to the second, one step at a time. **Each refinement step replaces one coarse data transaction with finer ones that together implement it.**

Level	Data transactions	Software structure introduced	Chip structure introduced	Principal obligations
Workload spec.	Required results: what, not how	Required computation, admitted inputs, operator semantics	—	Independent reference contract
0. Computation graph	Tensor values flowing along edges	Operators, edges, parameter and state versions	—	Correspondence to the specification
1. Operator transformation	Reductions and partial results inside each operator	Fusion, decomposition, reduction partitioning, algorithmic families	—	Operator semantics and dependencies preserved
2. Tensor execution mapping	Tile copies between memory levels; values held in slots	Tiles, layouts, placement, processor mapping	Register and on-chip memory capacity, number of units	Coverage, exact-once, address maps, lifetime
3. Instruction scheduling	Issued and completed transfers; synchronized reads	Instruction selection, transfers, synchronization	Instruction interface, transfer mechanisms, the boundary	Readiness, visibility, progress
4. System architecture	Transfers across units, channels and interconnect	Placement of work and data across units	Execution resources, memory hierarchy, interconnect	Resource compatibility, communication contracts
5. Microarchitecture	Requests served by banks, ports and queues	Bindings of operations to physical resources	Pipelines, banks, ports, queues, arbitration	Conflict handling, capacity, backpressure, progress

Table 3: Refinement levels. Each level makes one kind of data transaction finer and owes the obligations that the finer transactions can violate.

Every step has two consequences, and both stay small. The optimizer gains decisions: the finer transactions come with choices the coarse one did not have, some belonging to the software and some to the chip. The verifier owes an obligation: the finer transactions can fail in ways the coarse one could not express, and each step must prove that its new transactions still implement the coarse one. That proof is local to the step. Optimization and verification therefore move down the stack together, and neither ever faces the whole stack at once.

4.1 Refinement levels

Progressive refinement introduces software and chip structure in a fixed order of *what becomes concrete*, while leaving open *when* each choice is fixed. The workload contract is the reference; the remaining representations are indexed $\ell = 0, \ldots, 5$, with $\ell = 0$ the computation graph, so that level ℓ is layer $\ell + 2$ of §2.1. Table 3 names the data transaction that each level makes finer, the structure it introduces on both sides, and the obligations it owes.

Every level offers its own decisions: precision and batching at the workload; recomputation, operator order and memory planning on the graph; fusion, splitting and algorithm choice in the operator; tile shapes, loop order, layout, placement and buffer counts in the mapping, with register and shared-memory capacities and

unit counts on the chip side; instruction selection, software pipelining, asynchronous copies and barriers in the schedule, with the instruction interface on the chip side; the mix of units, capacities, channels and clock at the system level; and banks, ports, queue depths and arbitration in the microarchitecture. Chip choices can remain symbolic at early levels and become concrete later: a capacity introduced as a variable at level 2 constrains tiling there and is fixed only when the level-4 formulation selects a memory hierarchy. The software–hardware boundary is a decision of the same kind. For a mechanism j, a one-hot variable $b_{j\mu} \in \{0,1\}$, $\sum_\mu b_{j\mu} = 1$, selects whether it is realized as a software sequence, a programmable instruction mechanism or a dedicated controller; the modes share an external interface but differ in area, instruction overhead and the schedules they make legal.

Four kinds of change. Let the active formulation at iteration k be $\mathscr{F}_k = (\mathscr{R}_k, X_k, C_k, \hat{J}_k, \mathscr{D}_k)$, with representation, variables, constraints, modeled objective and a decoder $\mathscr{D}_k$ that turns a feasible assignment into a paired software implementation and chip configuration; its realizable set is $\mathrm{Real}(\mathscr{F}_k) = \mathscr{D}_k(\{x : C_k(x)\})$. The agent changes a formulation in four ways, and each carries its own obligation. *Vertical refinement* introduces a new object at the next level through a new template, and owes K1–K3 and composition (§5.4). *Representation expansion* adds a design family the model could not express, such as an online reduction or an asynchronous transfer mechanism, and owes certification of the new family's templates. *Horizontal reformulation* changes the encoding within a level while preserving decoded designs and objective values (auxiliary variables, cuts, symmetry breaking), and owes equivalence of the encoding in the style of reformulation checkers [11, 14]. *Restriction or cost revision* removes designs by a dominance rule or changes predicted costs, and owes a scope-stated dominance argument (§6.3) or nothing, if purely heuristic and labeled as such. These changes need not produce nested feasible sets, and the flow records each effect separately so that a bound proved for one formulation is never mistaken for a bound on another.

4.2 A worked progression: one GEMM, five steps finer

Consider a GEMM $Y = AB$ with dimensions M, N, K, where $Y_{ij} = \sum_k A_{ik} B_{kj}$. At the top, each output is a single coarse transaction: "sum these K products." Five templates make it concrete (Table 4), and each replaces one transaction with finer ones. The same five steps form the refinement chain whose checking cost is measured in §7.4.

Two of these steps show how a single line serves both readers. In step 2, with event times in a horizon $[0, H]$, a declared separation $\delta > 0$ and an order variable $z \in \{0,1\}$ for two values sharing storage,

$$l_v + \delta \le t_{v'} + (H + \delta)(1 - z), \qquad l_{v'} + \delta \le t_v + (H + \delta)\, z,$$

where t is a value's first destructive write and l its last required read. These inequalities are the solver's constraints and the verifier's lifetime precondition at once. Step 4 is the same idea specialized to rotating slots, and Listing 1 writes it out in full.

Read down the table, the optimization variables grow from a partition to a storage assignment, an event schedule, a slot rotation and a resource configuration, and the chip enters as soon as a step needs a capacity, a mechanism or a port. Read across it, every new freedom arrives with exactly the obligation it could violate, and none of these obligations can even be stated one step earlier: a lifetime is meaningless before there is storage, and a deadlock is meaningless before there are queues. Through every step, the family of admitted designs keeps a certified connection to the single GEMM at the top (Theorem 2, §5.4).

Step	Coarse transaction → finer	The optimizer gains	The verifier owes
1. Reduction partition	one sum of K products → partial sums over groups, then a combine	the assignment x_{kr} of each reduction index to a group	exact-once: every product contributes exactly once
2. Physical storage	a logical value → an occupancy of a reusable buffer	buffer assignments and the order z of values sharing a buffer	lifetime: a value survives until its last read
3. Asynchronous transfer	an atomic write → an issue event and a later completion	issue and completion times; the transfer mechanism $b_{j\mu}$	readiness: an operand is read only after its transfer completes
4. Double buffering	one buffer per value → generations rotating through S slots, loaded P tiles ahead	S, P, β, and the capacity that bounds them	generation correctness; lifetime becomes $P \leq S-2+\beta$ (Listing 1)
5. Shared resources	an untimed access → requests to banks, ports and bounded queues	bindings of requests to resources; queue depths and port counts on the chip	conflict freedom, no deadlock, progress under the declared arbitration

Table 4: One GEMM, five refinement steps. Each step adds decisions on both sides of the boundary and exactly the obligation it could violate.

5 Verification: Certifying Each Refinement

Verification proceeds with the refinement, one step at a time. At every step the verifier answers a single local question: do the finer transactions introduced here implement the coarser transaction they replace? It may assume that the level above has already been certified, and it owes nothing about the levels below, which have not yet been written. For the GEMM of §4.2 this means five separate, small proofs, none of which mentions the whole stack. Two results turn these local proofs into a global guarantee: composition (§5.4) chains them back to the workload, and incremental reuse (§5.6) keeps the chain cheap as formulations change.

Vertical, not horizontal. Reformulation checkers verify that two formulations at the same level are equivalent, typically that they have the same optimal solutions or that a solution of one maps to a solution of the other [11, 13, 14]. The obligation verified here differs in three ways. The two formulations sit at different levels, and the finer one contains objects (slots, transfers, queues) that the coarser one cannot name. Every admitted design, not just the optimal one, must compute what the coarser level demands; the objective plays no role in the proof. And the proof object is a backward map with a state relation and a progress rank, not a mapping between solutions.

5.1 Formulations as families of executable designs

At level ℓ, let d_ℓ collect the design variables and $C_\ell(p,d_\ell)$ the feasibility predicate assembled from the active templates. The formulation admits the family $F_\ell(p) = \{d_\ell \mid C_\ell(p,d_\ell)\}$, and the semantic claim concerns *every* admitted design, independently of which one the solver selects. Each formulation is stored as a typed model that declares variable domains, named constraint groups, the objective and an operation description P_ℓ. Substituting a design and an input into P_ℓ yields an executable system $\mathscr{T}_\ell(p,d_\ell,u) = \mathrm{Exec}_{\mathscr{A}_\ell}(P_\ell[p,d_\ell,u])$, where Exec is a fixed interpreter for the admitted operation constructors and $\mathscr{A}_\ell$ records the declared component and environment contracts, including the hardware's. A storage read returns the contents of the modeled location; it does not assume that the intended value is still there. Proving that correspondence is precisely a refinement obligation.

The system has states S_ℓ, initial states I_ℓ, terminal states T_ℓ without outgoing transitions, and a terminal observation O_ℓ. A transition $s \xrightarrow{w}_\ell s'$ emits a finite sequence w of visible events. Let $B_\ell(p, d_\ell, u)$ be the behaviors of all maximal executions, including failure or divergence. A formulation is *sound* if

$$\forall p \in \mathscr{P}\ \forall d_\ell \in F_\ell(p)\ \forall u \in U(p): \qquad B_\ell(p, d_\ell, u) \subseteq W(p, u).$$

The quantifiers matter. A proof for fixed shapes restricts $\mathscr{P}$, and testing sampled data does not establish the quantifier over $U(p)$ [105]. Floating-point accuracy is outside these obligations and is checked separately.

5.2 Backward maps and local obligations

A backward map $\pi_\ell(p, \cdot)$ interprets a level-$(\ell+1)$ design as a level-ℓ design, for example recovering logical tensors from tile and storage assignments. It must be total on feasible refined designs and preserve feasibility: $C_{\ell+1}(p, d) \Rightarrow C_\ell(p, \pi_\ell(p, d))$. The goal is behavior inclusion, $B_{\ell+1}(p, d, u) \subseteq B_\ell(p, \pi_\ell(p, d), u)$, derived from local conditions rather than assumed. Because the design map says nothing about intermediate values or progress, the template's certificate schema supplies a relation $R(s, a)$ between fine states s and coarse states a, and a rank $\rho(s, a) \in \mathbb{N}$ that measures progress during internal work. Writing $a \overset{m}{\underset{w\ c}{\Rightarrow}} a'$ for a path of exactly m coarse steps with visible events w, the certificate must establish four obligations:

- **Initialization.** $\forall s_0 \in I_f\ \exists a_0 \in I_c:\ R(s_0, a_0)$.
- **Step matching.** Every fine step has a finite coarse explanation with the same visible events, and a step explained by no coarse progress decreases the rank:

$$R(s,a) \wedge s \xrightarrow{w}_f s' \implies \exists a', m:\ a \overset{m}{\underset{w\ c}{\Rightarrow}} a' \wedge R(s', a') \wedge \big(m = 0 \Rightarrow \rho(s', a) < \rho(s, a)\big).$$

- **Progress.** A represented nonterminal state has a successor: $R(s,a) \wedge s \notin T_f \Rightarrow \exists s', w:\ s \xrightarrow{w}_f s'$.
- **Completion agreement.** $R(s,a) \wedge s \in T_f \Rightarrow a \in T_c \wedge O_f(s) = O_c(a)$.

The obligations quantify over all related states, which avoids relying on an unproved reachability approximation. They are generic; what changes from step to step of §4.2 is the relation R and which obligation carries the weight. For storage, R states that every buffer holds the logical value it currently stands for, and step matching for a read is exactly the lifetime condition. For asynchronous transfer, the coarse atomic write is matched to the transfer's completion, and step matching fails precisely when a read precedes completion, which is readiness. For double buffering, R tracks which generation each slot holds. For shared resources, step matching holds easily and the weight falls on progress: a waiting request must eventually be served, which the declared arbitration contract must justify.

5.3 Soundness of one refinement

Theorem 1 (local refinement soundness). Suppose the four obligations hold for a fixed design and input, and every maximal coarse execution terminates. Then every maximal fine execution terminates, and its visible history and terminal observation equal those of a maximal coarse execution. Hence $B_{\ell+1} \subseteq B_\ell$.

The proof (Appendix B) maps any fine execution to a concatenation of coarse paths with the same visible trace, and shows that an infinite fine execution would either map to an infinite coarse execution or decrease the rank forever. The rank permits finite silent work but excludes endless internal waiting. Progress that relies on hardware arbitration or transfer completion must follow from the declared hardware contract; an unstated

fairness assumption cannot supply it. Appendix B gives one sufficient contract, which bounds the number of consecutive transitions in which no outstanding event completes.

For the first refinement of §4.2, fine states record, for every output (i, j) and group r, the set D_{ijr} of consumed reduction indices and an accumulator b_{ijr}. The relation requires $b_{ijr} = \sum_{k \in D_{ijr}} A_{ik} B_{kj}$; the rank is $MNK - \sum |D_{ijr}| + 1$. Each consume step preserves the relation and lowers the rank, and finalization yields $Y_{ij} = \sum_r b_{ijr} = (AB)_{ij}$ because the groups partition the reduction. The certificate holds for every positive M, N, K, every partition and every matrix value. It establishes the partition refinement, but not that physical storage implements the atomic consume step; that is the next template's obligation.

5.4 Composition across the stack

Theorem 2 (progressive formulation soundness). Suppose the computation-graph formulation is sound with terminating workload behaviors, and every refinement satisfies feasibility preservation and the obligations of Theorem 1 for every admitted parameter, feasible fine design and admissible input, under compatible environment assumptions. Then

$$\forall p,\ \forall d \in F_L(p),\ \forall u \in U(p): \qquad B_L(p,d,u) \subseteq B_0\big(p, \Pi_{0:L}(d), u\big) \subseteq W(p,u).$$

The proof is an induction down the stack in which each step contributes only its own local certificate (Appendix B). Two caveats are essential. Soundness is distinct from nonemptiness, which requires a feasible witness at every level (K3). And a refinement may correctly restrict the design space without preserving its optimum: correctness composes, optimality does not, unless a reverse map or a dominance argument is supplied.

Theorem 2 is what licenses verifying one step at a time. Adding a sixth step costs one new local proof, not a new end-to-end argument. For the GEMM, the five local certificates compose into a proof that the design described in terms of banks, ports and queues computes $Y = AB$. When one step changes, only that step's certificate and those that depend on it need rechecking; along the five-step chain this cuts the cumulative solver queries at depth five from 92 to 27 (§7.4).

5.5 Checking contracts: the two-tier trust boundary

Tier 1: template soundness, once per template. For a new or revised template, the checker instantiates the obligations of §5.2 under the template's schema and scope. Witness substitution turns each obligation into an implication $H_j(\xi) \Rightarrow Q_j(\xi)$ over parameters, designs, states and events, and the checker searches for a violation, $\exists \xi : H_j(\xi) \wedge \neg Q_j(\xi)$. Arithmetic, array and finite-width conditions go to an SMT solver; symbolic reduction identities use library lemmas or proof-assistant proofs; finite execution families are explored exhaustively. Each query yields one of three outcomes. *Proved*: an unsatisfiable query establishes the implication within its declared scope, and the result is stored with its premises for reuse. *Counterexample*: a satisfying assignment is replayed from initialization by an independent interpreter; it establishes a workload defect only if the replay reconstructs an admissible execution that violates the workload contract, and otherwise shows a wrong map, an overly broad relation or an invalid progress argument, which the agent repairs in the certificate rather than the design. *Unresolved*: a timeout or unsupported theory leaves the template uncertified. Uncertainty is never treated as correctness.

Tier 2: instance check, once per candidate. The solver works with an exported predicate $\widehat{C}(p,z)$ over its own variables z, which may include auxiliaries, and a fixed decoder η maps solutions back to designs. Export is sound if $\forall p,z : \widehat{C}(p,z) \Rightarrow \exists d : \eta(p,z) = d \wedge C(p,d)$. A verified exporter proves this for a class of models;

Obligation	Depends on	Discharged by	Re-proved when
Lifetime	S, P, β (searched)	Constraint C3, proved sound once for the family	C3 changes, or a new searched decision touches slot reuse
Readiness	Leading barrier (frozen)	Frozen choice; family proof	The leading barrier is unfrozen
Coverage (M,N,K)	Grid rounding (frozen)	Frozen choice; symbolic proof per tile size	The grid rule is unfrozen
Exact-once (M,N,K)	Tile stride (frozen)	Frozen choice; symbolic proof per tile size	Overlapping tiles are admitted
No-phantom (M,N,K)	Guard (frozen)	Frozen choice; symbolic proof per tile size	The guard is removed or changed
Agreement	All of the above	Replay on a witness run	Any of the above

Table 5: The twelve instance obligations of the tiling template, what each depends on, and when it must be re-proved. Only lifetime needs a search constraint.

otherwise a translation check parses the emitted constraints and verifies the implication for that export. Each selected assignment is then checked against the exact declared constraints and integrality requirements. This is the gate inside the loop. Because K1 has already established that C with the frozen choices implies Pre, the per-candidate cost is that of evaluating a constraint system, not of re-proving correctness.

Which obligations need a constraint. The tiling template of Listing 1 has twelve instance obligations and one search constraint that is a precondition, C3. The two numbers differ because of the frozen/searched split (Table 5). Of the twelve obligations, only lifetime depends on a searched decision, so only it requires a search constraint; C3 is that constraint, and K1 holds for it by construction. The nine index obligations (coverage, exact-once and no-phantom, each over M, N and K) and readiness depend on choices the template froze, so they hold for every assignment the solver can produce and are proved once per tile size. Agreement is the replay. This is why the instance check costs about 25 ms: it evaluates C1–C3 on one assignment and reruns proofs whose premises are fixed. It also tells the agent which obligations to re-prove when it unfreezes a choice: unfreezing the leading barrier reopens readiness and nothing else, and the agent must then add a search constraint for readiness in the same way C3 serves lifetime. The register-tiling template of §2.4 unfreezes $a_{\max}$, which reopens nothing in Table 5 but adds the ownership obligation, discharged by a thread-to-tile map that the new template freezes.

The trusted base consists of the reviewed workload specification, the chosen proof logic and checker, the operation interpreter and the explicitly declared component and environment contracts. The agent may propose maps, invariants, preconditions and proof terms, but cannot turn an unproved statement into an axiom or silently change what an operation means. A new hardware assumption either follows from existing contracts or remains a visible, conditional premise.

5.6 Incremental reuse: keeping the gate affordable

Formulations change constantly in the flow, and rechecking everything after every change would make the gate the bottleneck. Instead, proofs are reused whenever their premises are unchanged. The proof library is a directed acyclic graph whose nodes are definitions, assumptions, theorem statements and checked proofs, and whose edges record dependence, including dependences hidden in imported lemmas, operation semantics and trusted backend assumptions. Constraints are kept as named groups, so that a lemma depends only on the groups it uses. After a change, the nodes whose definitions, assumptions, statements, certificates or incoming dependences changed, together with everything reachable from them, are invalidated and rechecked

Strength	Established by	May	If wrong
Proved bound	a one-sided bound, e.g. from monotonicity or an analytic roofline	*prune*: branch-and-bound cuts the branch and stays exact	cannot be, within its declared scope
Observed trend	measurement on the samples checked, e.g. "DRAM-bound on every probe so far"	*reorder*: exploration is prioritized, nothing is cut	the branch is reopened; no admitted design is affected

Table 6: The two strengths of a performance obligation.

in topological order.

Theorem 3 (soundness of incremental reuse). Suppose dependency information is complete, all affected nodes are rechecked against their revised statements and contexts, and acceptance requires accepted non-root premises. Then every retained or newly accepted theorem has a valid derivation under the revised root assumptions.

Corollary 1 (reuse under constraint strengthening). If parameter, design and input domains, the workload contract, execution semantics and environment assumptions are unchanged, and $C_{\text{new}} \Rightarrow C_{\text{old}}$, then soundness of the old formulation implies soundness of the new one.

Incremental checking saves work exactly when its bookkeeping overhead is smaller than the proof work it avoids; Appendix B gives the accounting. The two-tier boundary appears directly in it: template soundness is paid in the initial proof and the incremental rechecks, while the gate contributes only a per-candidate instance check.

5.7 Two kinds of obligation: correctness gates, performance steers

Everything above concerns *correctness* obligations: every admitted design computes the workload. The structures of §3 introduce a second kind. When a template relies on a regime, such as "this branch is DRAM-bound", the agent owes evidence that the regime holds. If that evidence fails, the design is not wrong; a compute-bound design still computes the right answer. What fails is the surrogate's ranking of candidates. In the terms of §6.1, a failed regime assumption is prediction or representation loss, never a violation of the workload contract. These are therefore *performance* obligations, and they behave differently in two ways. They never gate: a correctness counterexample is final, while a failed performance obligation reopens a branch, descends a level or revises the cost model, and discards nothing that was certified. And their strength decides what they may do (Table 6).

The rule is short enough to state once: **correctness obligations gate; performance obligations steer.** Whether a structural assumption may prune or may only prioritize depends on whether it was proved as a bound or merely observed as a trend. A regime claim is an inequality between resource times, and it holds over a branch B if $\min_{d \in B} t_{\text{DRAM}}(d) \geq \max_{d \in B} t_r(d)$ for every other resource r; because traffic is monotone in tile size and buffer count, this usually reduces to interval arithmetic. The inequality can also be written directly into C, so that the solver chooses only designs inside the regime and the assumption holds by construction, exactly as K1 makes Pre hold; designs outside belong to a sibling template with a different surrogate. When neither bound can be established, the agent probes sampled designs near the regime boundary, where the gain would be, and records the regime as a trend with a stated confidence: after n losing probes the fraction of the corner that would improve on the incumbent is below $3/n$ at 95% confidence, and the probe budget written into the run is what makes this behavior a specification rather than a hope. The guarantee the flow offers is therefore *regime-local optimality*: inside a regime with a proved bound the search is exact, and across

regimes it is exact over the regimes visited.

A ceiling prunes only if the cost model is *optimistic*: it must never predict a design faster than it measures. That property cannot be proved in general, but it can be checked continuously. Every evaluator measurement is compared with the model's prediction; while every measured design has run no faster than predicted, the model is one-sided on the evidence so far and its ceilings may prune. The first measurement that comes in faster than predicted demotes every ceiling-based pruning decision to a steer, and the pruned regimes are reopened. The full-network study of §7.3, in which the model erred by 58–63%, is the case this check is designed to catch.

6 Optimization: Searching Within the Contract

Progressive refinement is also what keeps the search tractable. The optimizer refines its formulation in lockstep with the verifier and never writes down the monolithic program of §1. It solves a *sequence* of formulations, each adding only the decisions that its finer transactions introduce. For the GEMM of §4.2, the first formulation chooses a reduction partition; the next adds buffer assignments, then transfer times and a mechanism, then a slot rotation, then bank, port and queue bindings. Every formulation in the sequence is small enough to solve. The tiling step of Listing 1 is a MILP with 93 variables that HiGHS solves to proven optimality in under 0.2 s (§7.2).

What keeps the sequence from becoming a greedy, layer-by-layer pass is how it treats coarser decisions. When a step begins, each decision made earlier is *fixed*, when nothing at the new step depends on it; *carried forward as a variable*, when it is coupled to the new decisions, as a tile shape is coupled to the shared-memory capacity that bounds it; or *reopened later*, when evidence shows that a coupled neighborhood is worth releasing. Chip parameters in particular may stay symbolic across several steps. The 14B study of §7.3 shows why the third option matters: a fusion decision made on the reference chip had to be reopened once the chip changed.

6.1 Why the formulation must evolve

A richer representation can only improve the best attainable design.

Proposition 2 (expansion under a fixed evaluator). If $\mathrm{Real}(\mathscr{F}_k) \subseteq \mathrm{Real}(\mathscr{F}_{k+1})$, both are nonempty and the true objective J is unchanged, then $\min_{\mathrm{Real}(\mathscr{F}_{k+1})} J \le \min_{\mathrm{Real}(\mathscr{F}_k)} J$.

A richer representation can also be much harder to search. The agent's job is to balance the two, and it is useful to separate the sources of design loss. Let $J^\star$ be the optimum over all correct designs, $J_k^\star(\mathrm{Safe})$ the optimum the active representation could reach with exact contracts, and $J_k^\star(\mathrm{Pre})$ the optimum under the contracts actually used.

Proposition 3 (four sources of design loss). Suppose the modeled objective is uniformly accurate on the realizable set, $|\hat{J}_k(d) - J(d)| \le \varepsilon_k$, and the selected design $\hat{d}_k$ satisfies $\hat{J}_k(\hat{d}_k) \le \min \hat{J}_k + \delta_k$. Then

$$J(\hat{d}_k) - J^\star \;\le\; \underbrace{J_k^\star(\mathrm{Safe}) - J^\star}_{\text{representation}} + \underbrace{\Delta_{\mathrm{pre}}}_{\text{contract}} + \underbrace{\delta_k}_{\text{optimization}} + \underbrace{2\varepsilon_k}_{\text{prediction}} .$$

Proof. For a true minimizer $d^\star$ under the contracts used, $J(\hat{d}_k) \le \hat{J}_k(\hat{d}_k) + \varepsilon_k \le \hat{J}_k(d^\star) + \delta_k + \varepsilon_k \le J_k^\star(\mathrm{Pre}) + \delta_k + 2\varepsilon_k$. Subtract $J^\star$ and split $J_k^\star(\mathrm{Pre}) - J^\star$ at $J_k^\star(\mathrm{Safe})$. □

Each term has its own remedy, and this is how the agent decides what to do next. Representation loss calls for a new family or template. Contract loss calls for a weaker, still sound precondition. Optimization error calls

for a stronger encoding or better search, and prediction error calls for a better cost model. More solver time cannot repair a prediction error, and a richer representation cannot repair a weak relaxation. A formal use of the bound requires a uniform error guarantee; residuals measured on sampled designs guide experiments but do not supply one.

6.2 Assembling the joint formulation

Each template contributes a block of variables and constraints. Blocks share chip variables, boundary variables and interface events, and their union is the level's mixed-integer program. Implementation selection is a set-cover over semantic units; chip supply is a one-hot configuration per hardware slot whose capacities enter resource constraints and whose area enters the budget; timing uses time-indexed start variables against per-tick capacity profiles; storage uses residency variables tied to production and last use, applied separately to registers, shared memory and external storage; and the worst-case objective is an epigraph variable over scenarios. Products of decisions are evaluated on finite menus or linearized exactly with tight bounds, never left as bilinear terms. A chip choice that changes a latency selects a corresponding performance mode; it cannot silently change a coefficient after optimization. The encodings are standard, and Appendix C writes them out; the tiling MILP of §7.2 shows the method on one template. For several scenarios, a master problem chooses the chip, boundary modes and algorithmic families, and scenario subproblems complete programs and schedules on that shared chip; a failed completion returns a no-good cut whose scope its derivation supports, and a compiler error or a timed-out schedule does not establish infeasibility. Bounds are indexed by the formulation that produced them, and a fitted cost prediction without a one-sided guarantee supports prioritization, not pruning (§5.7).

6.3 The agent's actions and how it chooses them

The agent acts on the formulation and on the population, and every action carries an explicit scope: which designs it adds or removes, which assumptions it changes, and which evaluation can resolve the next uncertainty. *Construction* writes or adapts templates, adds algorithmic families or introduces chip capabilities and boundary modes, guided by the structures of §3. *Reformulation* tightens bounds, breaks symmetries, adds valid inequalities or chooses a decomposition, and must preserve the decoded designs and their objective values. *Search* releases a coupled neighborhood, revisits a branch, selects a warm start or allocates an expensive evaluation.

Diagnosis. Before acting, the agent attributes every disagreement to one of five sources (Table 7). These responses are different, and treating every failure as another design sample wastes effort. A weak bound needs a stronger formulation, a fast solver repeatedly selecting slow designs needs a better cost model, and a refuted contract needs repair, not more search.

Coupled neighborhoods and conditional dominance. Many variables are consequences of a few structural decisions: a tile, a buffer organization, a boundary mode and a hardware allocation determine much of the remaining schedule. Useful neighborhoods release decisions connected by one mechanism; for a register-limited GEMM these are the tile shape, register capacity, resident thread groups and tensor-unit shape together. Releasing only the tile finds nothing if the enabling chip change stays fixed. Removing an alternative, likewise, is safe only in context.

Proposition 4 (contextual replacement). Let p and q be program fragments with the same functional interface, and Γ a set of chip configurations and surrounding programs. If every feasible design using p in Γ has a replacement using q that preserves the interface, remains feasible and is no worse, then removing p within Γ preserves the optimum over that domain.

Symptom	Source	Response
A selected design cannot be lowered or violates a realized limit	the formulation's constraints or interface	add the missing constraint or repair the interface
Measured ranking differs from predicted ranking	the predictive model	revise the cost mechanism
A promising family stays hard to explore despite an adequate model	the search policy or encoding	reformulate, decompose or change neighborhoods
The verifier refutes a template or an export	the contract	repair the template, its map or its certificate
The regime a template assumed no longer holds	a performance premise (§5.7)	reopen the branch, or descend one level with a new template

Table 7: Diagnosis: every disagreement is attributed to a source before the agent acts.

Every dominance fact is therefore stored with its context. A rule learned for one register budget does not apply outside it, and a change to the chip reopens the excluded alternative. When the replacement conditions are only suggested by measurement, the exclusion is a labeled heuristic, and the alternative remains in the population for reconsideration.

7 Evaluation

The evaluation has three parts, each asking one question. Does one template, written in the form of Listing 1, behave as the contract says (§7.2)? Does the flow, run on two hardware models, find designs that neither side of the stack finds alone (§7.3)? And is the gate affordable (§7.4)? Every number is labeled by the evidence behind it: *proved* within a stated bounded scope, *simulated* by complete instruction-level execution, *estimated* by calibrated aggregation of representative programs, *modeled* by a cost model alone, or *measured* on hardware.

7.1 Setup

Template prototype. The shared-memory tiling template of Listing 1 and the register-tiling template of §2.4 are implemented end to end. Their search faces are mixed-integer linear programs solved with HiGHS [91]; their correctness faces are discharged with the Z3 SMT solver [90], and every counterexample is replayed by an independent interpreter. Index obligations are proved for every tensor extent at each menu tile size, and schedule obligations for every interleaving of an event model with two warps and five reduction phases. The cost model belongs to the illustrative chip contract in the listing's scope (128 SMs, 1.8 TB/s, 80 TFLOP/s FP32, a 40-cycle barrier); it is not a measured chip, and the times it produces are modeled.

Refinement and workload checker. A Python/Z3 checker implements the backward maps and obligations of §5. Eight structural groups start from the same GEMM and add, in turn, reduction partitions, buffer reuse, asynchronous transfer completion, double buffering, address mapping, tiled boundaries, resource scheduling over banks, ports and bounded queues, and two GEMMs connected through a shared intermediate. Three workload components cover causal attention, prefill followed by two decode steps, and one training transition with two microbatches. Domains are small enough for every counterexample to be replayed from initialization and every finite state space to be explored completely. Cases and their expected classifications were fixed before the final runs.

Two hardware models. The co-design studies run on two models of two different chips.

Ventus is an open-source RISC-V GPGPU [89]. The reference chip has two streaming multiprocessors, each with up to eight resident warps of 32 threads, 1,024 vector-register slots, one (4, 8, 4) tensor unit, 128 KiB of shared memory and a 64 KiB L1 cache, with a 128 KiB shared L2 and one external-memory channel at 64 bytes per cycle. The search redistributes a fixed area budget across 27 hardware fields, from SM count and register ports to caches and load/store queues; external-memory service is held fixed, so any gain must come from using compute and bandwidth better. Area follows a library model with logic calibrated to 15 synthesis points in the ASAP7 predictive library [92] and SRAM costs from CACTI [56]; the small-chip budget is 1.11 mm^2. Software comes from the Ventus PyTorch backend compiled with Ventus LLVM, whose reference implementation already reuses operands across output tiles. Performance is measured in device cycles by an instruction-level timing model that follows compiled control flow and addresses and accounts for dependencies, ports, pipelines, queues, caches and external memory. On Ventus the chip itself is a decision: every hardware field in the menu is realizable in the model.

GB10 is a model of the NVIDIA GB10 in the DGX Spark: a Blackwell-generation GPU with 48 streaming multiprocessors sharing 128 GB of LPDDR5X with the CPU at a rated 273 GB/s, of which the GPU reaches 235 GB/s on reads. The model is mechanistic: schedulers and scoreboard, the load/store queue, shared-memory banks, asynchronous copy completion, the tensor pipe, register budget, L1, L2 and DRAM each have a mechanism whose parameters were measured by 40 microbenchmarks on the Spark, and the model was checked as a prediction against GEMM and attention kernels it was not built from, where it predicts compute phases within 5–15% and copy phases within 5–9%. The GB10 plays a different role from Ventus. Software decisions can be realized on the card, so for them the model's prediction is reported beside a measurement. Hardware decisions cannot be varied on the card; a study that changes SM count or shared-memory capacity in the GB10 model is extrapolating from the measured point, and its regime assumptions are recorded as *observed*, never *proved* (§5.7). Running the same studies on both models therefore tests two things at once: whether a co-design gain is a property of the workload or of one chip, and how a calibrated model and a cycle-level model of a chip with a design-time menu compare as evaluators.

Workloads, objectives and controls. The primary workload is Qwen2.5-0.5B-Instruct [93] at batch size one (24 layers, BF16 storage, FP32 accumulation); further studies use Qwen2.5-14B, a single-layer KV cache, and GPT-2 small [94] and Pythia-160M [95] for transfer. P128 and P512 denote prefill of a 128- or 512-token prompt, and D128 and D512 denote 16 decode steps from a KV cache of 128 or 512 tokens. The *cycle ratio* $r_w(d) = L_w(d)/L_w(d_{\text{ref}})$ is the latency objective of §2.1 normalized by the study's reference design; lower is better. Task A takes the worst ratio over the two prefill scenarios, task B over the two decode scenarios, and task C over all four. Every study reports the same final menus under five strategies, the *ladder*: the *baseline*; *software only*, with the chip fixed at the reference; *hardware only*, with the software fixed; *sequential*, which chooses the software on the reference chip and then the chip for that software; and *joint*, which solves one formulation over both. The ladder is reported on both models side by side.

What each study states. So that every study is an instance of the method rather than a separate experiment, each one states, before its numbers: the template(s) active, named as in Listing 1, with searched and frozen decisions listed; which frozen choice the study unfreezes, if any; what the gate certified for the selected design and the cost of the check; the evaluator used; and the cost model's prediction beside the evaluator's result.

7.2 One template, end to end

The tiling template is the paper's anchor result. It was hand-written, repaired by the agent and re-certified (§2.4); all times are modeled on the illustrative chip contract.

The precondition is exact. We ran the checker on all 24 combinations of slot count $S \in \{1,2,3\}$, prefetch dis-

Slots S	Trailing barrier	Safe prefetch distances P
1	yes	0
1	no	none
2	yes	0, 1
2	no	0
3	yes	0, 1, 2
3	no	0, 1

Table 8: Safe schedules of the shared-memory tiling template. The verifier's accepted set equals the precondition $P \leq S-2+\beta$ on all 24 combinations (proved, bounded).

GEMM	Selected $(T_m, T_n, T_k, S, P, \beta)$	Shared memory	Modeled time	Gate
4096^3	(128, 256, 16, 3, 1, 0)	72 KiB	1826.9 μs, memory-bound	12/12 proved
$64 \times 4096 \times 4096$	(64, 64, 16, 3, 1, 0)	24 KiB	75.1 μs, memory-bound	12/12 proved
1000^3, ragged tails	(128, 128, 32, 3, 1, 0)	96 KiB, at capacity	53.1 μs, compute-bound	12/12 proved

Table 9: Designs selected by the tiling template's MILP and certified by the gate. Times are modeled on the illustrative chip contract.

tance $P \in \{0, \ldots, 3\}$ and trailing barrier β. The set of schedules it certifies equals the template's precondition $P \leq S-2+\beta$ exactly, with no extra designs admitted and none wrongly excluded (Table 8). Every rejected combination fails the same obligation, lifetime. The result turns a piece of GPU programming folklore, that an extra buffer can replace the trailing `__syncthreads()`, into a checked condition: a third slot buys one more step of prefetch or the removal of the barrier, but not both.

Search and certification. Table 9 shows the designs the solver selects for three GEMM shapes. The MILP has 93 variables, 90 of them binary, and 248 constraints. HiGHS proves each selection optimal at zero gap in 0.03–0.16 s, and brute-force enumeration of the same 722 feasible designs reaches the same objective in every case. Certifying a selected design, twelve obligations, takes about 25 ms, for the reason Table 5 gives: only one obligation depends on what the solver chose. All three selections lie on the boundary of the precondition: three slots, prefetch distance one and *no* trailing barrier. The optimizer spends shared memory to remove a barrier, which is legal only because the precondition is the weakest one.

What a wrong contract costs. Replacing the template's precondition with a stronger or a weaker one, and keeping everything else fixed, reproduces both failure modes of §2.5 (Table 10). The stronger contract is safe but loses the optimum wherever barriers matter: 1.34% of latency on the compute-bound ragged shape, and nothing on the memory-bound shapes, so precondition loss depends on the workload as well as on the template. The weaker contract makes the solver choose, on all three shapes, double buffering with prefetch one and no trailing barrier, which the cost model predicts to be just as fast while using less shared memory. The gate rejects it and replays the race of Figure 4. The solver's answer is optimal for the constraints it was given, and wrong.

Precondition given to the solver	4096^3	64×4096^2	1000^3
Exact, $P \leq S-2+\beta$	optimal, certified	optimal, certified	optimal, certified
Too strong: always keep the trailing barrier	+0.0%, certified	+0.0%, certified	**+1.34%**, certified
Too weak: $P \leq S-1$, barrier ignored	**rejected by gate**	**rejected by gate**	**rejected by gate**

Table 10: Contract ablation. Percentages are precondition loss in modeled latency relative to the exact contract.

Variant	Failing obligation	Replayed defect	Violates
Floor-divided grid (tail uncovered)	coverage	missing reduction terms	frozen (grid)
Unguarded tail	no-phantom	write outside C; padding term added	frozen (guard)
Overlapping tiles (stride $T-1$)	exact-once	$C[127,0]$ receives $A[127,0]\,B[0,0]$ twice	frozen (disjoint)
Single buffer with prefetch one	lifetime	missing term	searched (C3)
Double buffer, $P=1$, no trailing barrier	lifetime	the race of Figure 4	searched (C3)
Missing leading barrier	readiness, lifetime	reads of uninitialized slot elements	unfrozen (barrier)

Table 11: Faults injected into the certified 1000^3 design, all rejected with a replayed defect.

Deliberately broken variants. Starting from the certified 1000^3 design, we injected six faults that a plausible implementation could contain. The gate rejects all six, each through the obligation responsible and each with a replayed defect (Table 11). The last column is the frozen/searched division of Table 5: three faults violate a frozen choice, which is what a code generator can break, and three violate a searched or unfrozen one, which is what a weak constraint lets the solver pick. For tiles overlapping with stride $T-1$, the coverage certificate fails without a semantic witness, and the checker correctly records that failure as unresolved; the real defect is found by the exact-once obligation that owns it.

The agent-written template. The register-tiling template of §2.4 is carried through the same three tables. [Safe-schedule table for the certified register-tiling template over its finite family; selected designs with $(T_m, T_n, T_k, S, P, \beta, r_m, r_n)$, solver time and obligations proved for the three GEMM shapes; and the fault table, including at least one fault against the new ownership obligation.] Two templates, one hand-repaired and one agent-written, both certified by the same checker, make the end-to-end claim about the method rather than about one example.

7.3 The flow on two hardware models

Five studies descend the stack in the order of §4.1. Each is run on both hardware models with the same menus and reports the ladder in one table, Ventus and GB10 side by side. The Ventus numbers draw on the project's public experiment reports [96, 97, 98, 99, 100]; the GB10 numbers come from the calibrated model and, for software-side decisions, from measurement on the Spark. Ratios are compared only within a study.

Study 1: kernel and queue co-design (0.5B). *Template:* shared-memory tiling (Listing 1) on the software side, with a queue-capacity template on the chip side whose searched decisions are the load/store and write-queue depths and whose frozen choice is the external-memory service. *Unfreezes:* nothing; the chip menu opens queue depths that the baseline treats as fixed. *Gate:* [obligations certified for the selected design and check time]. *Evaluator:* Ventus timing model (simulated for P128, extrapolated for the 512-token scenarios); GB10 model, with the software-only design measured on the Spark. On Ventus, compiled kernels that remove indexing and access overhead reduce P128 execution from 812.6 to 539.8 million cycles, and with the redistributed queues it falls to 264.1 million, a 67.5% reduction within 1.108 mm^2. The software cuts dynamic instructions from 386.3 to 334.4 million and requested bytes from 7.23 to 5.83 billion while leaving matrix instructions unchanged; the queues let the chip absorb the memory requests that remain. The combination beats both single-side controls on every task (Table 12), but joint search only ties sequential here: the benefit comes from letting both sides change, not from solving them together. [GB10 column; cost-model prediction beside the evaluator's result on both models; whether the queue finding (a write queue

	Ventus (simulated)					GB10 model ([evaluator])				
Task	Base	SW	HW	Seq	Joint	Base	SW	HW	Seq	Joint
A: prefill	1.000	0.758	0.440	0.377	0.377	1.000	[]	[]	[]	[]
B: decode	1.000	0.934	0.454	0.427	0.428	1.000	[]	[]	[]	[]
C: all	1.000	0.934	0.454	0.427	0.428	1.000	[]	[]	[]	[]

Table 12: Study 1, Qwen2.5-0.5B, compiled kernels with queue reallocation: worst-scenario cycle ratios (lower is better) under the five strategies, on both hardware models.

Design	Model	Area (mm^2)	P128	P512	D128	D512
Selected for tasks A and C	Ventus	1.081	0.342	0.331	0.295	0.272
Decode-specialized	Ventus	1.085	0.430	0.438	0.279	0.258
Selected for tasks A and C	GB10	—	[]	[]	[]	[]
Decode-specialized	GB10	—	[]	[]	[]	[]

Table 13: Study 2, Qwen2.5-0.5B, register tiling and tensor shape: cycle ratios of the selected designs. Ventus rows are simulated; GB10 rows are modeled, with the software-only kernel measured on the Spark.

the earlier model had missed, whose halving raises one subgraph's time by 49% on Ventus) has a counterpart in the GB10 load/store queue, whose depth the calibration measured.]

Study 2: register tiling and tensor shape (0.5B). *Template:* the register-tiling template of §2.4, with register-file size, tensor-unit shape and cache allocation as chip decisions. *Unfreezes:* the accumulator budget $a_{\max}$, which the tiling template froze; on the chip side, the register file that bounds it. *Gate:* [obligations and check time]. *Evaluator:* Ventus timing model, complete instruction simulation of all 24 layers and 16 decode steps; GB10 model, with the software-side kernels measured. This is the frozen/unfrozen story in one study. On Ventus, a 64×64 matrix tile cuts weight traffic by about one third but needs 173 vector registers. On the reference chip it makes the design worse (cycle ratio 0.362 to 0.438); once the register file, tensor shape and cache allocation are reopened, the same kernel becomes valuable. Packed query–key computation, unfavorable on the reference chip, likewise improves P512 from 0.377 to 0.334 on the new one. The final designs reach a 2.93× worst-prefill speedup, or 3.58× on the worst decode scenario for a decode-specialized chip that gives up prefill (Table 13). A kernel rejected on one chip is the right one on another, which is why the population keeps rejected alternatives with the conditions of their rejection (§3.1). [GB10 rows: on the card the register file is fixed at 64K registers per SM, so the software-only and sequential strategies are measurable and the hardware-side rows extrapolate; report the register-tile the solver selects for the fixed GB10 register file beside the one it selects when the file is a decision, and the model's prediction beside the measurement for the first.]

Study 3: fusion and mapping (14B). *Template:* a graph-level fusion template at level 1 whose fusion decision for the feed-forward gate and up projections is *frozen* in the sequential strategy, at the value chosen on the reference chip, and *searched* in the joint one; the mapping template below it carries the chip variables. *Unfreezes:* the fusion choice. *Gate:* [obligations and check time]. *Evaluator:* Ventus, estimated by calibrated aggregation of representative instruction simulations over all 48 layers on a chip scaled to 35.52 mm^2; GB10 model. On Ventus the software menu, solved to zero gap, includes shared input packing, grouped query heads, merged decode softmax and fused projections. The decisive choice is fusion: it is best on the reference chip but not on the selected one, which trades scalar registers and shared memory for more load/store and miss-status entries. The sequential strategy fixes the fusion choice on the reference chip and cannot undo it. Joint selection reopens it and lowers the decode objective by 4.10% relative to sequential, 4.43% relative to software only and 17.82% relative to hardware only (Table 14); for prefill the gain is 0.35%. A software

	Ventus (estimated)					GB10 model				
Task	Base	SW	HW	Seq	Joint	Base	SW	HW	Seq	Joint
A: prefill	1.000	0.996	0.968	0.964	0.961	1.000	[]	[]	[]	[]
B: decode	1.000	0.816	0.949	0.813	**0.780**	1.000	[]	[]	[]	[]
C: all	1.000	0.996	0.968	0.964	0.961	1.000	[]	[]	[]	[]

Table 14: Study 3, Qwen2.5-14B, fusion and mapping: worst-scenario cycle ratios on both models.

preference learned on one chip holds only under that chip. [GB10 column. On a chip whose shared memory is 228 KB per SM and whose register file is fixed, the model may or may not reverse the fusion preference; either result is reported, since the study's claim is about the sequential strategy's inability to reopen the choice, not about which way it goes.]

Study 4: KV-cache storage (decode). *Template:* a storage-format template at level 2 whose searched decisions are key and value precisions, quantization group size, head grouping and buffer reuse, and whose correctness precondition bounds the local attention error; the chip side searches 4,092 hardware proposals. *Unfreezes:* nothing. *Gate:* [obligations and check time]. *Evaluator:* Ventus timing prototype for one decode step of Qwen2.5-0.5B with a single-layer KV append; GB10 model with the format kernels measured. On Ventus, at allowed latency ratios of about 0.18–0.20, joint optimization needs 16–17% less storage than software only under 2% and 5% local error thresholds; at tighter ratios, software only has no feasible choice at all. Faster hardware lets a compact format meet the latency limit, although a hardware-first sequential search finds essentially the same chip. The accuracy result applies to local attention probes, not to full-model accuracy. [GB10 result: storage saved at the same latency ratios; whether the feasibility cliff appears at the same ratio.]

Study 5: full-network formulation, where the model is wrong. *Template:* the full-network formulation over two software families and 27 hardware fields, solved as one MILP. *Evaluator:* Ventus simulation; GB10 model and, for the software-only designs, measurement. This study is kept as the case in which the formulation itself is wrong. On Ventus, all nine MIP solves close with zero gap and predict a worst-case ratio near 0.333, but simulation returns 0.791–0.899, an underestimate of 58–63% (Table 15). The selections follow the error: the joint design is *worse* than hardware only on prefill (0.899 against 0.791). The solver did its job; the formulation did not describe the chip. A closed optimality gap certifies the model's optimum, not the design's. The same study on the GB10 model asks the paper's sharpest question about evaluators: [the GB10 model's error on the same formulation, beside Ventus's 58–63%; if the calibrated model's error is smaller, that is the argument for calibrated models; if not, the result is reported as it is]. The optimism check of §5.7 is what this study exercises: on Ventus the first measurement faster than predicted demotes every ceiling-based pruning decision to a steer.

Transfer. Running GPT-2 small and Pythia-160M on the Qwen-selected Ventus chips takes 3.07–5.60× their own baseline cycles in all 18 evaluations, mostly in the feed-forward down projection, where memory reads rise to 8.4–9.0× the baseline volume. A chip that improves Qwen is not a better language-model processor in general. The result supports storing knowledge with its scope, as the population requires (§3.1); it is not evidence that the population compounds, and the paper does not claim that it does. [Same transfer on the GB10-selected designs.]

What the ladder shows. Across the studies, changing the chip and the software together beats changing either alone, on Ventus in every study and on the GB10 model [in which studies]. The coupling matters more often than the search procedure: joint selection beats a sequential pass decisively only where an early commitment, a frozen choice, excludes the better design (study 3), and ties it where no such choice is in play

	Ventus (simulated)			GB10 model		
Strategy	A: prefill	B: decode	C: all	A: prefill	B: decode	C: all
Baseline	1.000	1.000	1.000	1.000	1.000	1.000
Software only	1.000	1.000	1.000	[]	[]	[]
Hardware only	0.791	0.797	0.827	[]	[]	[]
Sequential	0.791	0.797	0.827	[]	[]	[]
Joint	0.899	0.800	0.827	[]	[]	[]
MIP-predicted (all strategies)	≈ 0.333	≈ 0.333	≈ 0.333	[]	[]	[]

Table 15: Study 5, Qwen2.5-0.5B full-network optimization: evaluated cycle ratios beside the MIP's prediction, on both models.

Refinement	Accepted	New faults	Parent faults	Deadlocks	Other
Reduction partition	6	5	0	0	2 unresolved; 1 infeasible
Buffer reuse	5	4	1	0	1 infeasible
Transfer completion	5	4	1	0	1 unresolved; 1 infeasible
Double buffering	3	4	0	0	1 infeasible
Address mapping	4	5	1	0	1 unresolved
Tiled boundaries	3	5	0	0	—
Resource scheduling	3	2	0	2	1 unfair-stall control
Two connected GEMMs	3	2	0	1	—

Table 16: Outcomes for 78 structural variants (31 new faults, 3 deadlocks), proved on bounded instances. Unresolved entries are controls with deliberately incorrect certificates; none is counted as a defect.

(study 1). Where the two models disagree, [state which studies and in which direction], the disagreement is itself the evidence the population stores: a structure–regime pair that holds on one chip and not the other.

7.4 Verification cost

The gate runs inside the loop, so its cost is paid on every iteration. Two questions remain after the gate has been seen at work: does it catch the errors each level can introduce, and does its cost grow with what changed rather than with the depth of the stack?

Correctness across levels. Table 16 reports the 78 structural variants. Each refinement level introduces failures that the level above cannot express, and each is caught by the obligation that the new structure brings with it: storage breaks a correct computation when a buffer is overwritten between two reads of the same value; transfers separate issue from completion; all five boundary faults satisfy the coarse partition constraint yet omit tail work, duplicate it or touch unguarded padding; exhaustive exploration of 45–123 states per case finds two deadlocks among shared resources; and two GEMMs that pass their isolated checks produce a wrong result through an incorrect consumer address or a premature overwrite, and a shared resource adds a third deadlock. These are the cross-layer errors that layer-by-layer verification misses. The three workload components extend value identity beyond GEMM operands: all 100 valid schedules match their references, and every one of the 16 injected fault families (wrong mask, wrong key–value pairing, wrong cache extent, lost prefix, recomputation with the wrong random key, an omitted or duplicated gradient, an update made too early) yields a replayed counterexample. A matching tensor shape or storage slot does not identify a value; its position, version and computational origin must also agree.

Reuse down the refinement chain. The setting the flow depends on most is the five-step GEMM chain of

Depth	Level added	Full (ms)	Progressive (ms)	Queries (full / prog.)	Ratio
1	Reduction partition	25.5	26.3	8 / 8	0.97×
2	Storage	73.8	48.7	23 / 15	1.51×
3	Asynchronous transfer	289.0	200.6	43 / 20	1.50×
4	Double buffering	528.1	218.9	65 / 22	2.41×
5	Shared resources	958.4	380.6	92 / 27	2.51×

Table 17: Cumulative checking cost along the five-level GEMM refinement chain (measured checker time).

§4.2. *Full* mode rechecks every ancestor step at each depth; *progressive* mode keeps each accepted step's certificate once the fingerprints of its premises are confirmed unchanged. Neither caches individual queries, both use identical models, and faults injected at each step are rejected in both. Table 17 reports medians of seven paired repetitions. The gain grows with depth: at depth one there is nothing to inherit and bookkeeping costs a little; by depth five, progressive checking issues 27 solver queries instead of 92 and runs 2.51× faster. Across revision streams of the eleven structural and workload checkers, query-level reuse saves a further 1.22–2.06× without changing a classification (Appendix D).

Verification therefore stays tractable as the stack deepens: its cost tracks what changed, every proof is local, and every injected fault is caught with a replayed witness. The per-candidate gate on a certified template is a 25 ms check; the family proof is paid once.

8 Related Work

8.1 What others verify, what this paper proposes

Verification of programs, transformations and optimization models is well developed, and the right question to ask of each line of work is what object it verifies and over which family. Table 18 answers it.

Each of these methods verifies one artifact after it exists: a transformation, a rewrite, a reformulation, a solve, a mapping or a kernel. The kernel verifiers are the closest to our setting, and the contrast is sharpest there. ProofWright takes a CUDA kernel an LLM has already written and proves it memory-safe and race-free, one kernel at a time, verifying safety for about three quarters of KernelBench's level-1 kernels [103]; Volta proves one optimized ML kernel equal to a reference, soundly and, for a class of kernels, completely [104]; Model2Kernel finds memory-safety defects in the kernels an inference server actually calls [106]. All answer "is this kernel correct?" after search has chosen it, and the correctness-illusion study shows why the question must be asked at all: test-based acceptance on a few fixed shapes certifies buggy kernels as correct [105]. Our verifier answers a different question: "is every kernel this template can produce correct?" It is asked once, before the solver runs, and its answer is the constraint the solver must respect. A race the kernel verifiers would catch in the finished kernel is, in our flow, a design the solver cannot select, because the condition that excludes it, $P \leq S - 2 + \beta$, is the same line on both faces of the contract.

This difference in *when* is the paper's contribution to verification. Verifying after search leaves the search free to produce wrong designs and pays a proof per candidate. Verifying the family first pays one proof per template and a constant-time check per candidate (§5.5), and it feeds the verifier's knowledge back into the search as the weakest sound precondition (§2.5). The price is that the family must be expressible as a template, which is why §3 asks where templates come from. The refinement structure of §5 then composes these family-level certificates down the stack, so that the design the flow returns at the microarchitecture level carries a proof that reaches the workload. The foundations are classical: Dijkstra's weakest preconditions [12],

Line of work	Verifies	Family covered	When
Translation validation: Alive2, verified polyhedral codegen [40, 41, 42]	One compiler transformation preserves meaning	One input program, one output	After the transformation
Tensor rewrites: TensorRight, Tenspiler [43, 44]	One rewrite rule or one lifted expression	All tensor ranks for that rule	After the rewrite is written
Optimization models: CvxLean, FLARE, SOVER, EquivaMap [33, 11, 14, 13]	One reformulation equals another, or a solution satisfies its model	One formulation pair, at one level	After the reformulation
Solver certificates: VIPR, proof logging [35, 36]	The solver's answer is optimal for the model given	One solve	After the solve
Hardware interfaces: ILA, 3LA, BOLT, Kami [45, 46, 47, 48]	One mapping from compiler to accelerator against hardware semantics	One mapping, concrete parameters	After the mapping
Kernel verifiers: ProofWright, Volta, Model2Kernel [103, 104, 106]	One finished kernel is memory-safe and race-free, or equals a reference	One kernel, as written by a human, a compiler or an LLM	After the kernel exists
This paper	Every design a template admits computes the workload	The whole family, across one refinement step; composed down the stack	Before the solver selects, as a gate inside the loop

Table 18: What each line of verification work verifies, over which family, and when.

Abadi and Lamport's refinement mappings [37], Event-B's refinement with explicit proof obligations [38], and contract-based design [39]. Fine-grained caching of verification results [50, 51] and retrieval-augmented proving [52] make the incremental reuse of §5.6 practical.

8.2 Optimization, co-design and agents

Co-design and design-space exploration. A long line of work shows that decisions traditionally left to heuristics can be expressed mathematically: PLUTO selects loop transformations by integer linear optimization [28], CoSA encodes accelerator scheduling as a mixed-integer program [9], Checkmate optimizes rematerialization [15] and COSMA jointly optimizes scheduling, allocation and replacement for a scratchpad [16]. Joint search over mappings and hardware is equally established: DiGamma [17], DOSA [10], Timeloop [18], Gemmini [19], FireSim [20], Chipyard [83], Spatial [84], and in high-level synthesis SDC scheduling [87], ScaleHLS and AutoDSE [85, 86]. Component and system models such as Accelergy, McPAT, CACTI, Aladdin, gem5, GPGPU-Sim and Accel-Sim [54, 55, 56, 57, 58, 59, 60] supply the evaluation paths our hardware library rests on. Each of these systems optimizes within a formulation fixed in advance and confined to a few layers. Our flow treats the formulation itself as the object of design, grows it layer by layer, and certifies every step.

Tensor compilers and kernels. MLIR [21], TensorIR [22], Ansor [23], Mirage [24], Axon [25] and Exo [26, 27] determine which optimizations can be found by what the representation can express; cuBLAS, CUTLASS, Triton, TVM and FlashAttention [88, 75, 76, 79, 77, 78] are the software our designs must match; vLLM, Splitwise and DistServe [80, 81, 82] shape the serving workloads. All of them target a chip that is already fixed. In our flow, chip resources and the software–hardware boundary sit in the same representation as tiles, layouts and schedules, so a software alternative is ranked together with the hardware that would make it valuable.

Language models for optimization and design. OptiMUS constructs and solves models from problem descriptions [3], FormuEvo evolves MIP formulations toward solver efficiency [4], LLM4MIP analyzes formulated instances [29] and LLM4Branch discovers branching policies [32]; CompilerDream, Compiler-R1, ChatArch, graph placement and Dr. RTL reach the compiler and the chip [30, 31, 6, 7, 8]; KernelBench and the AI CUDA Engineer have language models write kernels [73, 74]; AlphaEvolve and FunSearch couple a model with automatic evaluation [1, 2], and evolving agent ensembles accumulate reusable mechanisms [5]. These methods generate a program, a formulation or a design for one problem at one layer and judge it by solver outcome or measured quality, with no correctness certificate. In our flow the model generates templates, which carry their own correctness conditions and scope, and nothing it produces is accepted until the verifier certifies it; §2.4 is the gap this fills.

Synthesis, counterexamples and libraries. A template with holes that a solver fills is close in spirit to program sketching [64] and syntax-guided synthesis [61]; stochastic superoptimization searches instruction sequences directly [62]. Counterexample-guided inductive synthesis and abstraction refinement [64, 63] and Benders decomposition with its logic-based and combinatorial variants [65, 66, 67] turn failures into constraints. DreamCoder, LILO and Voyager grow libraries of reusable abstractions [68, 69, 70], MAP-Elites maintains a population diverse in behavior [71], and evolution through large models uses a language model as the mutation operator [72]. A template is a sketch whose holes span the chip and the software, whose correctness conditions are also the solver's constraints, and whose population is a library of formulation knowledge, kept diverse by structure and admitted only once certified.

9 Limits and Next Steps

What the evidence supports. A template's search constraints and its correctness conditions can coincide, so that the solver's optimum is correct by construction (§7.2). A language model can write a template in that form and the verifier can decide it: one template was repaired and re-certified, and one was written and [certified after *n* repair rounds] (§2.4). Each per-step formulation is small enough to solve to proven optimality. Verification decomposes into local proofs that catch the errors each step introduces and compose along the stack, at a cost that tracks what changed (§7.4). And on two hardware models, the coupled designs that this structure exposes beat designs in which either side is held fixed (§7.3).

What it does not. The limits are stated once, here.

- *Template writing.* Two templates were written or repaired by the agent, each in one run. The claim is that the verifier decides, not that the agent writes correct templates; how often it does, across the catalog and across refinement steps, is not measured. The structure catalog is drawn from the studies in which its structures were exploited by hand.
- *The population.* The population has not been run across problems. The one transfer measurement is negative (§7.3); it supports storing knowledge with its scope and is not evidence that the population compounds.
- *Fidelity.* A closed optimality gap certifies the model's optimum, not the design's; study 5 shows a 58–63% prediction error on Ventus. The tiling cost model's chip contract is illustrative, and the GB10 model extrapolates whenever a hardware decision moves it off the measured card.
- *Scope of the proofs.* Proofs cover bounded shapes and finite schedule families, and rely on a trusted checker written in Python over Z3. The schedule obligations are proved on a two-warp, five-phase event model; the argument that two warps suffice for lifetime (any race is between one writer and one reader, and the other warps are bystanders) is stated but not mechanized, and a deadlock that needs three parties is outside it. The workload components are not yet connected to the structural storage and

resource models by checked mappings.

- *Scope of the measurements.* The Ventus studies are cycle-based and area-constrained, with no power model and no physical implementation. No design is compared against an industrially optimized implementation such as cuBLAS or FlashAttention. Physical and technology effects are outside the paper.

Next steps. Three experiments follow directly. Let the agent write templates across the catalog and the five GEMM steps, and report for each the verifier's verdict, the repair rounds, the precondition loss of the certified ones and how much of the search each structure removes. Reuse one certified template across new workloads and chip contracts, and compare the cost of an instance check with that of re-deriving the template, which turns the two-tier boundary into a measured saving. And carry one certified design through to execution on the DGX Spark against cuBLAS, so that the cost model's optimism record of §5.7 is kept against a real card.

The lesson of this work is that the obstacle to whole-stack co-design is less the size of the design space than the absence of a formulation in which that space can be searched and trusted. Progressive refinement provides the structure. Templates make each step both solvable and checkable, the physics of the stack tells the agent which template to write, and the verifier, asked before the solver runs rather than after, is what lets the agent's writing be trusted.

A The Structure Catalog

Table 19 lists the structures of §3, with the origin and effect of each. The list is open.

Structure	Origin	Effect on the search
Bottleneck dominance	one resource (e.g. DRAM) is far costlier than the rest	within its regime, that resource's utilization is a near-sufficient statistic; many objectives collapse to one tight bound, and branch-and-bound acquires optimal substructure
Near-decomposability [101]	interactions across the stack are local	the coupling matrix is block-banded; clusters are solved jointly and only the weak coupling between them is paid for. The hierarchy itself cuts chains: shared memory separates DRAM from the registers, so a three-way search becomes two pairwise ones
Concavity	performance saturates in every resource (the roofline knee [102])	optima lie on constraint boundaries or at a few knees; search the knees, not the continuum
Monotonicity	at fixed area, more of a resource never hurts	valid one-sided bounds, which is what exact pruning needs
Scenario decomposition	prefill and decode share a chip but want different software	the chip is the complicating variable; once fixed, each scenario is an independent subproblem (Benders structure, §6.2)
Low effective dimension	only the knobs touching the bottleneck matter	twenty coupled decisions become the three or four that move bytes; the rest are frozen or coarsened
Symmetry	identical units, interchangeable banks, mirrored layouts	symmetry breaking removes factorial blocks at no cost
Discrete menus	powers of two, a handful of tile shapes	domains are small and enumerable
Dominance	most designs are dominated on every metric	the Pareto front is thin; dominated designs are skipped without evaluation

Table 19: Exploitable structures in the co-design space.

B Proofs and the Hardware Progress Contract

Proof of Theorem 1. Initialization and repeated step matching map any fine execution to a concatenation of coarse paths with the same visible trace. A finite maximal fine execution ends in a terminal state, by progress, and completion agreement gives equal observations. An infinite fine execution would either map to an infinite coarse execution, contradicting coarse termination, or eventually make no coarse progress, in which case the rank would decrease forever, which is impossible in $\mathbb{N}$. □

A sufficient hardware progress contract. Progress that relies on hardware arbitration or transfer completion must follow from the declared hardware contract. One sufficient contract bounds the number of consecutive primitive transitions in which no outstanding event completes. With $n_e(s)$ outstanding events and a countdown $h(s) \le \beta_{\max}$, the potential $\Phi(s) = (\beta_{\max}+1)\,n_e(s) + h(s)$ decreases on every transition and bounds execution length by $(\beta_{\max}+1)N_e + \beta_{\max}$.

Proof of Theorem 2. Define coarser designs backward by $d_\ell = \pi_\ell(p, d_{\ell+1})$. Feasibility preservation gives $C_0(p,d_0)$. Soundness of the graph formulation gives termination and $B_0 \subseteq W$, and induction over levels with Theorem 1 gives $B_{\ell+1} \subseteq B_\ell$ at every level. □

Incremental reuse. The proof library is a DAG $G = (V,E)$; each node v carries a certificate $\kappa_v : \Gamma_v \vdash P_v$. After a change producing $G' = (V',E')$, let Δ be the nodes whose definitions, assumptions, statements, certificates or incoming dependences changed, and invalidate $A = (\Delta \cup \mathrm{Reach}^{+}_{G\cup G'}(\Delta)) \cap V'$, rechecked in topological order. Theorem 3 follows by induction over that order: every retained node's premises are either roots or

accepted nodes whose derivations are valid by the induction hypothesis. Incremental checking saves work exactly when its overhead h_j is smaller than the proof work it avoids,

$$T_j^{\text{inc}} < T_j^{\text{full}} \iff h_j < \sum_{i \in Q_j \setminus A_j} t_{i,j},$$

where Q_j are the obligations after refinement j, $A_j \subseteq Q_j$ those new or invalidated, and $t_{i,j}$ the cost of obligation i. The total accounted cost of a design campaign is $T_{\text{total}} = V_{\text{base}} + \sum_j T_j^{\text{inc}} + n\,c_{\text{check}} + V_{\text{lower}}$, covering initial proof construction, incremental rechecking, n per-candidate instance checks and the remaining implementation verification.

C Assembling the Joint MILP

Indices are u for semantic units, r for candidate regions with implementations $c \in C_r$, i for hardware slots with configurations $p \in P_i$, and h for resource types.

Coverage and selection. With $q_{rc} \in \{0,1\}$ selecting an implementation, $\sum_{r,c:\,u \in \text{cover}(r,c)} q_{rc} = 1$ for all u and $\sum_{c \in C_r} q_{rc} \le 1$ for all r. Algebraically different families, such as a materialized softmax and an online recurrence, are selected first by a family variable; each family then activates its own internal decomposition.

Chip supply and capability. With $y_{ip} \in \{0,1\}$ selecting at most one configuration per slot, $\text{Cap}_h(y) = \sum_{i,p} K_{iph}\, y_{ip}$ and $A(y,b) = A_0 + \sum_{i,p} A_{ip}\, y_{ip} + \sum_{j,\mu} A^B_{j\mu}\, b_{j\mu} \le A_{\max}$; an implementation that needs capability g requires $q_{rc} \le \sum_{i,p:\, g \in \text{cap}(i,p)} y_{ip}$.

Timing and shared resources. With $x_{rct} = 1$ when implementation (r,c) starts at tick t, $\sum_t x_{rct} = q_{rc}$, a duration d_{rc} and a resource profile U_{rch}, $\sum_{r,c,t} U_{rch}(\tau - t)\, x_{rct} \le \text{Cap}_h(y)$ for all h, τ. For a producer r and consumer r' linked by $g_{rr'}$, $s_{r'} \ge f_r + \delta_{rr'} - H(1 - g_{rr'})$.

Storage and ports. With residency variables $\lambda_{v\tau}$ tied to production and last use, $\sum_v B_v\, \lambda_{v\tau} + \sum_{r,c,t} S_{rc}(\tau - t)\, x_{rct} \le \text{Cap}_{\text{mem}}(y)$ for all τ, applied separately to registers, shared memory and external storage. Port demand and outstanding transfers consume separate resources.

Objective and linearizations. The worst-case objective is an epigraph variable, $T \ge L_w / L_w^{\text{ref}}$ for all w. For binaries q, y, $z = q \wedge y$ uses $z \le q$, $z \le y$, $z \ge q + y - 1$; for a binary q and a continuous $v \in [L,U]$, $z = qv$ uses $Lq \le z \le Uq$ and $v - U(1-q) \le z \le v - L(1-q)$. Tight bounds matter: a generic large constant yields a valid but needlessly weak relaxation.

Movable boundaries. A family that needs asynchronous completion requires $q_{rc} \le \sum_{\mu \in \mathcal{M}_{rc}} b_{j\mu}$, together with a compatible transfer engine, enough in-flight slots and a buffer lifetime spanning completion. Choosing a mode also selects its lowering recipe: a programmable mode emits completion events and waits, a controller mode emits configuration and invocation commands. Specialization pays a shared area cost once; software pays a recurring instruction cost wherever it is used.

D Reuse Across Revision Streams

The GEMM checker caches proved queries keyed by formula, source hash and solver settings, over streams containing a baseline, relevant edits, an injected fault and its repair. The workload checker caches whole groups of schedules once every member has passed, over streams that also expand the family and force a context change. Paired runs alternate their order, start from an empty cache, and agree on every classification (Table 20). One attention pair ran slower with reuse (11.6 ms against 9.4 ms), and the structural streams reuse exact queries rather than automatically extracted semantic dependencies. The three workload components

accept 100 valid schedules (36 attention, 48 prefill-and-decode, 16 training) and reject 16 fault families (3, 6 and 7), each with one unresolved control and one infeasible control.

Stream	Full (ms)	With reuse (ms)	Paired ratio	Reused
Reduction partition	168.6	104.8	1.65×	26/41 queries
Buffer reuse	138.4	103.6	1.34×	29/57
Transfer completion	198.6	159.9	1.24×	39/77
Double buffering	645.6	480.7	1.35×	45/77
Address mapping	1127.3	923.3	1.22×	59/101
Tiled boundaries	368.5	202.3	1.79×	29/49
Resource scheduling	70.1	48.7	1.44×	33/52
Two connected GEMMs	79.3	53.2	1.44×	55/84
Causal attention	10.5	6.4	1.65×	17/30 groups
Prefill and decode	37.1	18.1	2.06×	71/120
Training transition	30.3	15.1	2.01×	11/20

Table 20: Verification reuse over revision streams (median of three paired repetitions).

References

[1] B. Romera-Paredes et al. *Mathematical Discoveries from Program Search with Large Language Models*. Nature, 2024.

[2] A. Novikov et al. *AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery*. arXiv:2506.13131, 2025.

[3] A. AhmadiTeshnizi et al. *OptiMUS-0.3: Using Large Language Models to Model and Solve Optimization Problems at Scale*. arXiv:2407.19633, 2026.

[4] H. Yuan et al. *FormuEvo: LLM-Guided Evolution for Discovering Solver-Efficient Mixed-Integer Programming Formulations*. arXiv:2608.23353, 2026.

[5] Yu and Yang. *Evolving Ensemble of Agents*. arXiv:2605.09018, 2026.

[6] Wu et al. *ChatArch: A Knowledge-driven Graph-of-thought LLM Framework for Processor Architecture Optimization*. ACM TODAES 31(2), 2025.

[7] A. Mirhoseini et al. *A Graph Placement Methodology for Fast Chip Design*. Nature 594:207–212, 2021.

[8] Fang et al. *Dr. RTL: Autonomous Agentic RTL Optimization through Tool-Grounded Self-Improvement*. arXiv preprint, 2026.

[9] Q. Huang et al. *CoSA: Scheduling by Constrained Optimization for Spatial Accelerators*. ISCA, 2021.

[10] C. Hong, Q. Huang, G. Dinh, M. Subedar, and Y. S. Shao. *DOSA: Differentiable Model-Based One-Loop Search for DNN Accelerators*. MICRO, 2023.

[11] H. Robbins, C. Lawless, M. Udell, and E. Vitercik. *FLARE: Verifying MILP Reformulations with LLM-Based Theorem Proving*. arXiv:2608.25220, 2026.

[12] E. W. Dijkstra. *Guarded Commands, Nondeterminacy and Formal Derivation of Programs*. Communications of the ACM, 18(8):453–457, 1975.

[13] H. Zhai, C. Lawless, E. Vitercik, and L. Leqi. *EquivaMap: Leveraging LLMs for Automatic Equivalence Checking of Optimization Formulations*. arXiv:2502.14760, 2025.

[14] S. Bhattacharyya and M. Baranwal. *SOVER: Formal Certification of Optimization Reformulations via LLM-Assisted SMT Verification*. arXiv:2609.00728, 2026.

[15] P. Jain et al. *Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization*. MLSys, 2020.

[16] Y. Li, A. Gupta, and S. Malik. *Combined Scheduling, Memory Allocation and Tensor Replacement for Minimizing Off-Chip Data Accesses of DNN Accelerators*. arXiv:2311.18246, 2023.

[17] S.-C. Kao et al. *DiGamma: Domain-aware Genetic Algorithm for HW-Mapping Co-optimization for DNN Accelerators*. DATE, 2022.

[18] A. Parashar et al. *Timeloop: A Systematic Approach to DNN Accelerator Evaluation*. ISPASS, 2019.

[19] H. Genc et al. *Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration*. DAC, 2021.

[20] S. Karandikar et al. *FireSim: FPGA-Accelerated Cycle-Exact Scale-Out System Simulation in the Public Cloud*. ISCA, 2018.

[21] C. Lattner et al. *MLIR: Scaling Compiler Infrastructure for Domain Specific Computation*. CGO, 2021.

[22] S. Feng et al. *TensorIR: An Abstraction for Automatic Tensorized Program Optimization*. ASPLOS, 2023.

[23] L. Zheng et al. *Ansor: Generating High-Performance Tensor Programs for Deep Learning*. OSDI, 2020.

[24] M. Wu et al. *Mirage: A Multi-Level Superoptimizer for Tensor Programs*. OSDI, 2025.

[25] A. Kothari, S. Zhu, D. Kroening, and C. Sung. *Axon: A Synthesizing Superoptimizer for Tensor Programs*. arXiv:2606.26344, 2026.

[26] Y. Ikarashi et al. *Exo 2: Growing a Scheduling Language*. ASPLOS, 2025.

[27] D. Z. Akeley, Y. Ikarashi, and J. Ragan-Kelley. *Exo-GPU: Safe, Imperative, User-schedulable Programming for Tensor Cores*. arXiv:2609.16389, 2026.

[28] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. *A Practical Automatic Polyhedral Parallelizer and Locality Optimizer*. PLDI, 2008.

[29] Y. Huang, W. Gao, D. Ge, M. Udell, and Y. Ye. *How Much Can LLMs Help Solve MIPs?* Research website and released workflows, 2026. `https://llm4mip.github.io/`

[30] Deng et al. *CompilerDream: Learning a Compiler World Model for General Code Optimization*. KDD, 2025.

[31] Pan et al. *Compiler-R1: Towards Agentic Compiler Auto-tuning with Reinforcement Learning*. arXiv preprint, 2025.

[32] Hou et al. *LLM4Branch: Large Language Model for Discovering Efficient Branching Policies of Integer Programs*. arXiv preprint, 2026.

[33] A. Bentkamp, R. Fernández Mir, and J. Avigad. *Verified Reductions for Optimization*. TACAS, 2023.

[34] Ö. Akgün, A. M. Frisch, I. P. Gent, C. Jefferson, I. Miguel, and P. Nightingale. *Conjure: Automatic Generation of Constraint Models from Problem Specifications*. IJCAI, 2023.

[35] K. K. H. Cheung, A. Gleixner, and D. E. Steffy. *Verifying Integer Programming Results*. IPCO, 2017.

[36] A. Hoen, A. Oertel, A. Gleixner, and J. Nordström. *Certifying MIP-based Presolve Reductions for 0–1 Integer Linear Programs*. CPAIOR, 2024.

[37] M. Abadi and L. Lamport. *The Existence of Refinement Mappings*. Theoretical Computer Science, 82(2):253–284, 1991.

[38] Rodin Project. *Rodin User's Handbook, Section 2.7.4: Refinement*.

[39] P. Nuzzo, A. L. Sangiovanni-Vincentelli, D. Bresolin, L. Geretti, and T. Villa. *A Platform-Based Design Methodology With Contracts and Related Tools for the Design of Cyber-Physical Systems*. Proceedings of the IEEE, 103(11):2104–2132, 2015.

[40] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr. *Alive2: Bounded Translation Validation for LLVM*. PLDI, 2021.

[41] N. Courant and X. Leroy. *Verified Code Generation for the Polyhedral Model*. POPL, 2021.

[42] X. Li, H. Liang, and X. Feng. *Verified Validation for Affine Scheduling in Polyhedral Compilation*. TASE, 2024.

[43] J. Arora et al. *TensorRight: Automated Verification of Tensor Graph Rewrites*. POPL, 2025.

[44] J. Qiu, C. Cai, S. Bhatia, N. Hasabnis, S. A. Seshia, and A. Cheung. *Tenspiler: A Verified Lifting-Based Compiler for Tensor Operations*. ECOOP, 2024.

[45] B.-Y. Huang, H. Zhang, P. Subramanyan, Y. Vizel, A. Gupta, and S. Malik. *Instruction-Level Abstraction (ILA): A Uniform Specification for System-on-Chip (SoC) Verification*. arXiv:1801.01114, 2018.

[46] B.-Y. Huang et al. *Application-Level Validation of Accelerator Designs Using a Formal Software/Hardware Interface*. arXiv:2203.00218, 2023.

[47] A. Gaonkar et al. *Verification of Compiler-to-Accelerator Mappings for Machine Learning Accelerators*. arXiv:2609.30651, 2026.

[48] J. Choi, M. Vijayaraghavan, B. Sherman, A. Chlipala, and Arvind. *Kami: A Platform for High-Level Parametric Hardware Specification and Its Modular Verification*. ICFP, 2017.

[49] R. Mukherjee, S. Joshi, J. O'Leary, D. Kroening, and T. Melham. *COVERIF: Hardware/Software Co-verification Using Path-based Symbolic Execution*. arXiv:2001.01324, 2020.

[50] K. R. M. Leino and V. Wüstholz. *Fine-grained Caching of Verification Results*. CAV, 2015.

[51] W. Visser, J. Geldenhuys, and M. B. Dwyer. *Green: Reducing, Reusing and Recycling Constraints in Program Analysis*. FSE, 2012.

[52] K. Yang et al. *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*. NeurIPS, 2023.

[53] X. Leroy. *Formal Verification of a Realistic Compiler*. Communications of the ACM, 52(7):107–115, 2009.

[54] Y. N. Wu, J. S. Emer, and V. Sze. *Accelergy: An Architecture-Level Energy Estimation Methodology for Accelerator Designs*. ICCAD, 2019.

[55] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi. *McPAT: An Integrated Power, Area, and Timing Modeling Framework for Multicore and Manycore Architectures*. MICRO, 2009.

[56] N. Muralimanohar, R. Balasubramonian, and N. P. Jouppi. *CACTI 6.0: A Tool to Model Large Caches*. HP Laboratories Technical Report HPL-2009-85, 2009.

[57] Y. S. Shao, B. Reagen, G.-Y. Wei, and D. Brooks. *Aladdin: A Pre-RTL, Power-Performance Accelerator Simulator Enabling Large Design Space Exploration of Customized Architectures*. ISCA, 2014.

[58] N. Binkert et al. *The gem5 Simulator*. ACM SIGARCH Computer Architecture News, 39(2), 2011.

[59] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt. *Analyzing CUDA Workloads Using a Detailed GPU Simulator*. ISPASS, 2009.

[60] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers. *Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling*. ISCA, 2020.

[61] R. Alur et al. *Syntax-Guided Synthesis*. FMCAD, 2013.

[62] E. Schkufza, R. Sharma, and A. Aiken. *Stochastic Superoptimization*. ASPLOS, 2013.

[63] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. *Counterexample-Guided Abstraction Refinement*. CAV, 2000.

[64] A. Solar-Lezama, L. Tancau, R. Bodík, S. Seshia, and V. Saraswat. *Combinatorial Sketching for Finite Programs*. ASPLOS, 2006.

[65] J. F. Benders. *Partitioning Procedures for Solving Mixed-Variables Programming Problems*. Numerische Mathematik, 4:238–252, 1962.

[66] J. N. Hooker and G. Ottosson. *Logic-Based Benders Decomposition*. Mathematical Programming, 96:33–60, 2003.

[67] G. Codato and M. Fischetti. *Combinatorial Benders' Cuts for Mixed-Integer Linear Programming*. Operations Research, 54(4):756–766, 2006.

[68] K. Ellis et al. *DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning*. PLDI, 2021.

[69] G. Grand et al. *LILO: Learning Interpretable Libraries by Compressing and Documenting Code*. ICLR, 2024.

[70] G. Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. TMLR, 2024.

[71] J.-B. Mouret and J. Clune. *Illuminating Search Spaces by Mapping Elites*. arXiv:1504.04909, 2015.

[72] J. Lehman et al. *Evolution through Large Models*. arXiv:2206.08896, 2022.

[73] A. Ouyang et al. *KernelBench: Can LLMs Write Efficient GPU Kernels?* ICML, 2025.

[74] R. T. Lange et al. *The AI CUDA Engineer: Agentic CUDA Kernel Discovery, Optimization and Composition*. Sakana AI technical report, 2025.

[75] NVIDIA. *CUTLASS: CUDA Templates for Linear Algebra Subroutines*. Software library.

[76] P. Tillet, H. T. Kung, and D. Cox. *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations*. MAPL, 2019.

[77] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022.

[78] T. Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR, 2024.

[79] T. Chen et al. *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*. OSDI, 2018.

[80] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023.

[81] P. Patel et al. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting*. ISCA, 2024.

[82] Y. Zhong et al. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI, 2024.

[83] A. Amid et al. *Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs*. IEEE Micro, 40(4):10–21, 2020.

[84] D. Koeplinger et al. *Spatial: A Language and Compiler for Application Accelerators*. PLDI, 2018.

[85] H. Ye et al. *ScaleHLS: A New Scalable High-Level Synthesis Framework on Multi-Level Intermediate Representation*. HPCA, 2022.

[86] A. Sohrabizadeh, C. H. Yu, M. Gao, and J. Cong. *AutoDSE: Enabling Software Programmers to Design Efficient FPGA Accelerators*. ACM TODAES, 27(4), 2022.

[87] J. Cong and Z. Zhang. *An Efficient and Versatile Scheduling Algorithm Based on SDC Formulation*. DAC, 2006.

[88] NVIDIA. *cuBLAS: Basic Linear Algebra on NVIDIA GPUs*. Software library.

[89] Ventus GPGPU Project. *Ventus: An Open-Source RISC-V GPGPU*. Tsinghua University. `https://github.com/THU-DSP-LAB/ventus-gpgpu`.

[90] L. de Moura and N. Bjørner. *Z3: An Efficient SMT Solver*. TACAS, 2008.

[91] Q. Huangfu and J. A. J. Hall. *Parallelizing the Dual Revised Simplex Method*. Mathematical Programming Computation, 10:119–142, 2018.

[92] L. T. Clark et al. *ASAP7: A 7-nm FinFET Predictive Process Design Kit*. Microelectronics Journal, 53:105–115, 2016.

[93] Qwen Team. *Qwen2.5 Technical Report*. arXiv:2412.15115, 2024.

[94] A. Radford et al. *Language Models are Unsupervised Multitask Learners*. OpenAI technical report, 2019.

[95] S. Biderman et al. *Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling*. ICML, 2023.

[96] Ventus Co-Design Project. *Compiled Kernels and Queue Allocation for Qwen2.5-0.5B*. Public experiment report, issue 2, October 9, 2026. `https://github.com/BinaryCheater/ventus-codesign/issues/2`.

[97] Ventus Co-Design Project. *Tiling, Register Allocation and Attention-Mapping Co-Design Study*. Public experiment report, issue 3, October 9, 2026. `https://github.com/BinaryCheater/ventus-codesign/issues/3`.

[98] Ventus Co-Design Project. *Qwen2.5-14B Joint Hardware–Software Optimization*. Public experiment report, issue 4, October 9, 2026. `https://github.com/BinaryCheater/ventus-codesign/issues/4`.

[99] Ventus Co-Design Project. *Latency-Constrained KV Storage*. Public experiment report, issue 5, October 9, 2026. `https://github.com/BinaryCheater/ventus-codesign/issues/5`.

[100] Ventus Co-Design Project. *Qwen Full-Network Optimization and GPT-2/Pythia Transfer*. Public experiment report, issue 6, October 9, 2026. `https://github.com/BinaryCheater/ventus-codesign/issues/6`.

[101] H. A. Simon. *The Architecture of Complexity*. Proceedings of the American Philosophical Society, 106(6):467–482, 1962.

[102] S. Williams, A. Waterman, and D. Patterson. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. Communications of the ACM, 52(4):65–76, 2009.

[103] B. Chatterjee, D. Zagieboylo, S. Damani, S. Hari, and C. Kozyrakis. *ProofWright: Towards Agentic Formal Verification of CUDA*. arXiv:2511.12294, v2, March 2026.

[104] B. Driscoll, K. Dubey, A. Wei, N. Kayal, R. Sharma, and A. Aiken. *Equivalence Checking of ML GPU Kernels*. OOPSLA, 2026 (arXiv:2511.12638).

[105] D. Sarkar. *The Correctness Illusion in LLM-Generated GPU Kernels*. arXiv:2606.20128, 2026.

[106] M. He, S. Xia, H. Jia, W. Wu, and L. Song. *Model2Kernel: Model-Aware Symbolic Execution for Safe CUDA Kernels*. arXiv:2603.24595, 2026.